Slip 1

Q.1) Create the following database in 3NF using PostgresSQL. [Total Marks:10]

Consider the following Bank database which maintains information about its branches, customers and

their loan applications.

Branch (Bid integer, brname varchar (30), brcity varchar (10))

Customer (Cno integer, cname varchar (20), caddr varchar (35), city varchar (15))

Loan_application (Lno integer, l_amt_required money, lamtapproved money, l_date date)

Relationship:

Branch, Customer, Loan_application are related with ternary relationship as follows:

Ternary (Bid, Cno, Lno)

Constraints: Primary key, l_amt_required should be greater than zero.

Using above database solve following Queries:

1. Display sum of loan amount approved branch wise from 1st June 2019 to 1st June 2020.

2. Display customers details of 'Aundh' branch.

3. Display total of loan amount required.

4. Display all details of Branch table.

Q.2) Using above database solve following questions [Total Marks: 20]

1. Write a trigger before insert record of customer. If the customer number is less than or

equal to zero and customer name is null then give the appropriate message [10]

2. Write a stored function to accept customer name as an input parameter and display

loan information of that customer. [10]

Answer :

```
--Create Tables

-- Branch Table
CREATE TABLE Branch (
    Bid INT PRIMARY KEY,
    brname VARCHAR(30),
    brcity VARCHAR(10)
);

-- Customer Table
CREATE TABLE Customer (
    Cno INT PRIMARY KEY,
    cname VARCHAR(20),
    caddr VARCHAR(35),
    city VARCHAR(15)
);

-- Loan Application Table
CREATE TABLE Loan_application (
    Lno INT PRIMARY KEY,
    l_amt_required NUMERIC(12,2) CHECK (l_amt_required > 0),
    l_amt_approved NUMERIC(12,2),
    l_date DATE
);
```

```
-- Ternary Relationship Table
CREATE TABLE Ternary (
    Bid INT,
    Cno INT,
    Lno INT,
    PRIMARY KEY (Bid, Cno, Lno),
    FOREIGN KEY (Bid) REFERENCES Branch(Bid),
    FOREIGN KEY (Cno) REFERENCES Customer(Cno),
    FOREIGN KEY (Lno) REFERENCES Loan_application(Lno)
);

--Insert  Records
INSERT INTO Branch VALUES
(1,'Aundh','Pune'),
(2,'ShivajiNagar','Pune'),
(3,'Camp','Pune'),
(4,'Baner','Pune'),
(5,'Wakad','Pune'),
(6,'Kothrud','Pune'),
(7,'Hadapsar','Pune'),
(8,'Pimpri','Pune');

INSERT INTO Customer VALUES
(101,'Rahul','Aundh Road','Pune'),
(102,'Priya','Baner Road','Pune'),
(103,'Amit','MG Road','Pune'),
```

(104,'Neha','Wakad Road','Pune'),

(105,'Karan','Kothrud','Pune'),

(106,'Sneha','Camp Area','Pune'),

(107,'Vikas','Hadapsar Road','Pune'),

(108,'Pooja','Pimpri Colony','Pune');

INSERT INTO Loan_application VALUES

(201,50000,45000,'2019-07-10'),

(202,70000,65000,'2019-09-15'),

(203,30000,25000,'2020-01-20'),

(204,90000,85000,'2020-04-18'),

(205,60000,55000,'2021-02-12'),

(206,75000,70000,'2019-12-05'),

(207,40000,35000,'2020-05-30'),

(208,80000,76000,'2022-03-25');

INSERT INTO Ternary VALUES

(1,101,201),

(2,102,202),

(3,103,203),

(4,104,204),

(5,105,205),

(6,106,206),

(7,107,207),

(8,108,208);

--Sum of loan amount approved branch wise between 1 June 2019 and 1 June 2020

```
SELECT b.brname,
SUM(l.l_amt_approved) AS total_approved
FROM Branch b
JOIN Ternary t ON b.Bid = t.Bid
JOIN Loan_application l ON t.Lno = l.Lno
WHERE l.l_date BETWEEN '2019-06-01' AND '2020-06-01'
GROUP BY b.brname;
```

Display customers details of 'Aundh' branch

```
SELECT c.*
FROM Customer c
JOIN Ternary t ON c.Cno = t.Cno
JOIN Branch b ON t.Bid = b.Bid
WHERE b.brname = 'Aundh';
```

--Display total loan amount required

```
SELECT SUM(l_amt_required) AS total_required
FROM Loan_application;
```

--Display all branch details

```
SELECT * FROM Branch;
```

```
--Q2) Trigger Before Insert on Customer
--Function for Trigger

CREATE OR REPLACE FUNCTION check_customer()
RETURNS TRIGGER AS
$$
BEGIN
IF NEW.Cno <= 0 OR NEW.cname IS NULL THEN
RAISE EXCEPTION 'Customer number must be greater than zero and name
cannot be NULL';
END IF;
RETURN NEW;
END;
$$
LANGUAGE plpgsql;


--Create Trigger

CREATE TRIGGER customer_validation
BEFORE INSERT
ON Customer
FOR EACH ROW
EXECUTE FUNCTION check_customer();
```

--Test Trigger

--Wrong Record

INSERT INTO Customer VALUES (0,NULL,'Pune','Pune');

--Output :ERROR: Customer number must be greater than zero and name cannot be NULL

--Correct Record

INSERT INTO Customer VALUES (109,'Ram','Wakad','Pune');

--Stored Function

--Function to display loan info by customer name

```
CREATE OR REPLACE FUNCTION get_loan_info(cust_name VARCHAR)
RETURNS TABLE(
    customer_name VARCHAR,
    loan_no INT,
    loan_required NUMERIC,
    loan_approved NUMERIC,
    loan_date DATE
)
AS
$$
BEGIN
RETURN QUERY
```

```
SELECT c.cname,
    l.Lno,
    l.l_amt_required,
    l.l_amt_approved,
    l.l_date
FROM Customer c
JOIN Ternary t ON c.Cno = t.Cno
JOIN Loan_application l ON t.Lno = l.Lno
WHERE c.cname = cust_name;
END;
$$
LANGUAGE plpgsql;

--Call Function

SELECT * FROM get_loan_info('Rahul');
```

Slip 2

Q.1) Create the following database in 3NF using PostgresSQL. [Total Marks: 10]

Consider the following Bank database which maintains information about its branches, customers

and their loan applications.

Branch (Bid integer, brname varchar (30), brcity varchar (10))

Customer (Cno integer, cname varchar (20), caddr varchar (35), city varchar (15))

Loan_application (Lno integer, l_amt_required money, lamtapproved money, l_date date)

Relationship:

Branch, Customer, Loan_application are related with ternary relationship as follows:

Ternary (Bid, Cno, Lno)

Constraints: Primary key, l_amt_required should be greater than zero.

Using above database solve following Queries:

1. Insert 5 records into Branch table.

2. Display loan details from the ‘Pimpri’ branch.

3. Display name of customer whose name start by letter ‘A’.

4. Display customer details who have applied for a loan of 8, 00,000.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Create a view to list all customer detail of ‘Pimpri’ branch. [10]

2. Write a stored function to count number of customers of particular branch. (Accept branch

name as an input parameter). Display message for invalid branch name. [10]

Answer :

```
--create tables
-- Branch Table
CREATE TABLE Branch(
    Bid INT PRIMARY KEY,
    brname VARCHAR(30),
    brcity VARCHAR(10)
);

-- Customer Table
CREATE TABLE Customer(
    Cno INT PRIMARY KEY,
    cname VARCHAR(20),
    caddr VARCHAR(35),
    city VARCHAR(15)
);

-- Loan Application Table
CREATE TABLE Loan_application(
    Lno INT PRIMARY KEY,
    l_amt_required NUMERIC(12,2) CHECK(l_amt_required > 0),
    l_amt_approved NUMERIC(12,2),
    l_date DATE
);
```

```
-- Ternary Relationship Table
CREATE TABLE Ternary(
    Bid INT,
    Cno INT,
    Lno INT,
    PRIMARY KEY(Bid,Cno,Lno),
    FOREIGN KEY (Bid) REFERENCES Branch(Bid),
    FOREIGN KEY (Cno) REFERENCES Customer(Cno),
    FOREIGN KEY (Lno) REFERENCES Loan_application(Lno)
);

--Insert Records
-- Insert 5 Records into Branch Table

INSERT INTO Branch VALUES
(1,'Pimpri','Pune'),
(2,'Aundh','Pune'),
(3,'Baner','Pune'),
(4,'ShivajiNagar','Pune'),
(5,'Camp','Pune'),
(6,'Wakad','Pune'),
(7,'Kothrud','Pune'),
(8,'Hadapsar','Pune');
```

```
INSERT INTO Customer VALUES
(101,'Amit','Pimpri Road','Pune'),
(102,'Anita','Baner Road','Pune'),
(103,'Rahul','Aundh Road','Pune'),
(104,'Sneha','Camp Area','Pune'),
(105,'Ajay','Kothrud','Pune'),
(106,'Pooja','Wakad','Pune'),
(107,'Akash','Hadapsar','Pune'),
(108,'Neha','Shivaji Nagar','Pune');

INSERT INTO Loan_application VALUES
(201,500000,450000,'2020-05-10'),
(202,800000,750000,'2021-06-12'),
(203,300000,250000,'2019-09-18'),
(204,600000,550000,'2022-01-11'),
(205,200000,180000,'2020-03-05'),
(206,400000,350000,'2021-07-20'),
(207,700000,650000,'2019-11-15'),
(208,100000,90000,'2023-02-25');

INSERT INTO Ternary VALUES
(1,101,201),
(1,102,202),
(2,103,203),
(3,104,204),
(4,105,205),
(5,106,206),
```

```
(6,107,207),
(7,108,208);


--Insert 5 Records into Branch Table (Already done above)

SELECT * FROM Branch LIMIT 5;


-- Display Loan Details from 'Pimpri' Branch

SELECT l.*
FROM Loan_application l
JOIN Ternary t ON l.Lno = t.Lno
JOIN Branch b ON t.Bid = b.Bid
WHERE b.brname = 'Pimpri';


-- Display Customers whose name starts with 'A'

SELECT *
FROM Customer
WHERE cname LIKE 'A%';
```

```
-- Display customer details who applied for loan of 8,00,000

SELECT c.*
FROM Customer c
JOIN Ternary t ON c.Cno = t.Cno
JOIN Loan_application l ON t.Lno = l.Lno
WHERE l.l_amt_required = 800000;



--Q2  Create View for Customers of Pimpri Branch

CREATE VIEW pimpri_customers AS
SELECT c.*
FROM Customer c
JOIN Ternary t ON c.Cno = t.Cno
JOIN Branch b ON t.Bid = b.Bid
WHERE b.brname = 'Pimpri';

--Test View
SELECT * FROM pimpri_customers;

-- Stored Function to Count Customers of a Branch
CREATE OR REPLACE FUNCTION count_customers(branch_name
VARCHAR)
RETURNS TEXT
AS
$$
DECLARE
```

```
    total INT;
BEGIN

SELECT COUNT(*) INTO total
FROM Customer c
JOIN Ternary t ON c.Cno = t.Cno
JOIN Branch b ON t.Bid = b.Bid
WHERE b.brname = branch_name;

IF total = 0 THEN
    RETURN 'Invalid Branch Name or No Customers Found';
ELSE
    RETURN 'Total Customers in ' || branch_name || ' branch: ' || total;
END IF;

END;
$$
LANGUAGE plpgsql;

-- Run Stored Function
SELECT count_customers('Pimpri');
```

Slip 3

Q.1) Create the following database in 3NF using PostgresSQL.

Consider the following database of Student Competition [Total Marks: 10]

Student (Sid,sname,class)

Competition (cno,cname, ctype)

Relationship:

Student and Competition related with many to many relationship with attribute rank and year.

Constraints: Primary key sid,cno.

Using above database solve following Queries: [10]

1. Display Student id and name.

2. List the name of student in sorting order.

3. List the name of students who took part in national level competition.

4. List the competition details..

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger before inserting record of student in student table. If the student id is

less than or equal to zero then display the appropriate error message. [10]

2. Create a View to display student details along with their Competition name. [10]

Answer:

```
-- Create Tables

-- Student Table

CREATE TABLE Student(

   Sid INT PRIMARY KEY,

   sname VARCHAR(30),
```

```
    class VARCHAR(10)
);

-- Competition Table
CREATE TABLE Competition(
    cno INT PRIMARY KEY,
    cname VARCHAR(40),
    ctype VARCHAR(20)
);

-- Relationship Table (Many-to-Many)
CREATE TABLE Participation(
    Sid INT,
    cno INT,
    rank INT,
    year INT,
    PRIMARY KEY(Sid,cno),
    FOREIGN KEY (Sid) REFERENCES Student(Sid),
    FOREIGN KEY (cno) REFERENCES Competition(cno)
);


-- Insert Records

INSERT INTO Student VALUES
(1,'Amit','FYBCA'),
(2,'Sneha','SYBCA'),
```

(3,'Rahul','TYBCA'),
(4,'Anita','FYBBA'),
(5,'Karan','SYBBA'),
(6,'Pooja','TYBBA'),
(7,'Akash','FYBCS'),
(8,'Neha','SYBCS');

INSERT INTO Competition VALUES
(101,'Coding Challenge','National'),
(102,'Debate Competition','College'),
(103,'Quiz Contest','State'),
(104,'Hackathon','National'),
(105,'Sports Meet','College'),
(106,'Science Fair','State'),
(107,'Math Olympiad','National'),
(108,'Poster Presentation','College');

INSERT INTO Participation VALUES
(1,101,1,2023),
(2,102,2,2023),
(3,103,3,2022),
(4,104,1,2023),
(5,105,2,2022),
(6,106,3,2023),
(7,107,1,2024),
(8,108,2,2024);

```
-- Display Student ID and Name

SELECT Sid, sname
FROM Student;


-- List Student Names in Sorting Order

SELECT sname
FROM Student
ORDER BY sname;


-- List Students who took part in National Level Competition

SELECT s.sname
FROM Student s
JOIN Participation p ON s.Sid = p.Sid
JOIN Competition c ON p.cno = c.cno
WHERE c.ctype = 'National';
```

```
-- List Competition Details
SELECT *
FROM Competition;



-- Q2 Trigger (Before Insert Student)
--Trigger Function

CREATE OR REPLACE FUNCTION check_student_id()
RETURNS TRIGGER AS
$$
BEGIN
IF NEW.Sid <= 0 THEN
RAISE EXCEPTION 'Student ID must be greater than zero';
END IF;
RETURN NEW;
END;
$$
LANGUAGE plpgsql;



-- Create Trigger
CREATE TRIGGER student_validation
BEFORE INSERT
ON Student
FOR EACH ROW
EXECUTE FUNCTION check_student_id();
```

```
-- Test Trigger Wrong Insert
INSERT INTO Student VALUES (0,'Test','FYBCA');

-- output
Student ID must be greater than zero

-- Create View (Student + Competition Name)

CREATE VIEW student_competition_view AS
SELECT s.Sid, s.sname, s.class, c.cname
FROM Student s
JOIN Participation p ON s.Sid = p.Sid
JOIN Competition c ON p.cno = c.cno;

-- View
SELECT * FROM student_competition_view;
```

Slip 4

Q1) Create the following database in 3NF using PostgresSQL. [Total Marks: 10]

Consider the following database of Bus-Transport System. Many buses run on one route. Drivers

are allotted to buses shift-wise.

Bus (Bus_no int , capacity int , depot_name varchar (20))

Route (Route_no int, source varchar (20), destination varchar (20), no_of_stations int)

Driver (Driver_no int, driver_name varchar (20), license_no int, address varchar

(20),age int , salary float)

Relationship:

Bus and Route related with many to one relationship.

Bus and Driver related with many to many relationship with descriptive attributes, Shift – it can

be 1 (Morning) or 2 (Evening) and Date_of_duty_allotted.

Constraints: Primary key, license_no must be unique, Bus capacity should not be null.

Using above database solve following Queries:

1. Display details of all the drivers having age more than 40.

2. List driver name having salary > 30,000.

3. Insert 2 records into route table.

4. Display driver name who drive bus no ‘12’.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger before inserting the driver record in driver table, if the age is between 18 and

25, then display error message ‘Invalid input’. [10]

2. Write a stored function to display details of buses running on route_no = ‘ ’. (Accept

route_no as an input parameter.)

Answer:

```
-- Create Tables

-- Route Table
CREATE TABLE Route(
    Route_no INT PRIMARY KEY,
    source VARCHAR(20),
    destination VARCHAR(20),
    no_of_stations INT
);

-- Bus Table (Many buses on one route)
CREATE TABLE Bus(
    Bus_no INT PRIMARY KEY,
    capacity INT NOT NULL,
    depot_name VARCHAR(20),
    Route_no INT,
    FOREIGN KEY(Route_no) REFERENCES Route(Route_no)
);

-- Driver Table
CREATE TABLE Driver(
    Driver_no INT PRIMARY KEY,
    driver_name VARCHAR(20),
    license_no INT UNIQUE,
```

```
    address VARCHAR(20),
    age INT,
    salary FLOAT
);

-- Bus and Driver Many-to-Many Relationship
CREATE TABLE Bus_Driver(
    Bus_no INT,
    Driver_no INT,
    shift INT CHECK (shift IN (1,2)),
    date_of_duty_allotted DATE,
    PRIMARY KEY(Bus_no,Driver_no),
    FOREIGN KEY(Bus_no) REFERENCES Bus(Bus_no),
    FOREIGN KEY(Driver_no) REFERENCES Driver(Driver_no)
);

-- Insert Records
INSERT INTO Route VALUES
(1,'Pune','Mumbai',12),
(2,'Pune','Nashik',10),
(3,'Pune','Satara',8),
(4,'Pune','Kolhapur',15),
(5,'Pune','Solapur',11),
(6,'Pune','Aurangabad',13),
(7,'Pune','Ahmednagar',9),
(8,'Pune','Lonavala',6);
```

```
INSERT INTO Bus VALUES
(10,50,'Swargate',1),
(11,45,'Shivajinagar',2),
(12,60,'Swargate',3),
(13,55,'Pimpri',4),
(14,50,'Hadapsar',5),
(15,48,'Katraj',6),
(16,52,'Pimpri',7),
(17,46,'Shivajinagar',8);
```

```
INSERT INTO Driver VALUES
(101,'Ramesh',1001,'Pune',45,35000),
(102,'Suresh',1002,'Pune',38,28000),
(103,'Mahesh',1003,'Pune',50,42000),
(104,'Amit',1004,'Pune',32,31000),
(105,'Rajesh',1005,'Pune',41,33000),
(106,'Vijay',1006,'Pune',36,27000),
(107,'Anil',1007,'Pune',48,36000),
(108,'Sunil',1008,'Pune',29,29000);
```

```
INSERT INTO Bus_Driver VALUES
(10,101,1,'2024-01-10'),
(11,102,2,'2024-01-11'),
(12,103,1,'2024-01-12'),
(12,104,2,'2024-01-13'),
(13,105,1,'2024-01-14'),
(14,106,2,'2024-01-15'),
(15,107,1,'2024-01-16'),
(16,108,2,'2024-01-17');


-- Display drivers having age more than 40
SELECT *
FROM Driver
WHERE age > 40;


-- List driver name having salary > 30000
SELECT driver_name
FROM Driver
WHERE salary > 30000;


-- Insert 2 records into Route table
INSERT INTO Route VALUES
(9,'Pune','Nagpur',18),
(10,'Pune','Goa',20);
```

```
-- Display driver name who drives bus no 12
SELECT d.driver_name
FROM Driver d
JOIN Bus_Driver bd ON d.Driver_no = bd.Driver_no
WHERE bd.Bus_no = 12;


-- Q2 Trigger (Before Insert Driver)
-- Trigger Function

CREATE OR REPLACE FUNCTION check_driver_age()
RETURNS TRIGGER AS
$$
BEGIN
IF NEW.age BETWEEN 18 AND 25 THEN
RAISE EXCEPTION 'Invalid input';
END IF;
RETURN NEW;
END;
$$
LANGUAGE plpgsql;


-- Create Trigger
CREATE TRIGGER driver_age_validation
```

```
BEFORE INSERT
ON Driver
FOR EACH ROW
EXECUTE FUNCTION check_driver_age();



-- Test Trigger
INSERT INTO Driver VALUES (109,'Test',1010,'Pune',20,20000);

-- Output
ERROR: Invalid input

-- Stored Function (Display buses running on given route)
-- Stored Function to display buses running on given route

CREATE OR REPLACE FUNCTION buses_on_route(r_no INT)
RETURNS TABLE(
   bus_no INT,
   capacity INT,
   depot_name VARCHAR(20)
)
AS
$$
BEGIN
   RETURN QUERY
   SELECT b.Bus_no, b.capacity, b.depot_name
   FROM Bus b
```

```
    WHERE b.Route_no = r_no;
END;
$$
LANGUAGE plpgsql;


-- Run Stored Function
SELECT * FROM buses_on_route(3);
```

Slip 5

Q.1) Create the following database in 3NF using PostgresSQL. [Total Marks: 10]

Consider the following database of Bus-Transport System. Many buses run on one route. Drivers

are allotted to the buses shift-wise.

Bus (Bus_no int , capacity int , depot_name varchar(20))

Route (Route_no int, source varchar(20), destination varchar (20), no_of_stations int)

Driver (Driver_no int , driver_name varchar(20), license_no int, address varchar (20),age

int , salary float)

Relationship:

Bus and Route related with many to one relationship.

Bus and Driver related with many to many relationship with descriptive attributes, Shift – it can

be 1 (Morning) or 2 (Evening) and Date_of_duty_allotted.

Constraints: Primary key, license_no must be unique, Bus capacity should not be null.

Using above database solve following Queries:

1. Display the drivers name.

2. Display the route details on which buses of capacity 40 runs.

3. Insert 2 records into route table.

4. Display bus detail of Rout_no=4.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Create a View to Display driver details whose duty allotted on 26-9-2013. [10]

2. Write a function using cursor to display all the dates on which a driver has driven a bus

(Accept the driver name as an input parameter) [10]

Answer:

```
-- Create Tables
-- Route Table
CREATE TABLE Route(
    Route_no INT PRIMARY KEY,
    source VARCHAR(20),
    destination VARCHAR(20),
    no_of_stations INT
);

-- Bus Table
CREATE TABLE Bus(
    Bus_no INT PRIMARY KEY,
    capacity INT NOT NULL,
    depot_name VARCHAR(20),
    Route_no INT,
    FOREIGN KEY(Route_no) REFERENCES Route(Route_no)
);

-- Driver Table
CREATE TABLE Driver(
    Driver_no INT PRIMARY KEY,
    driver_name VARCHAR(20),
    license_no INT UNIQUE,
    address VARCHAR(20),
    age INT,
```

```
    salary FLOAT
);

-- Bus-Driver Relationship Table
CREATE TABLE Bus_Driver(
    Bus_no INT,
    Driver_no INT,
    shift INT CHECK (shift IN (1,2)),
    date_of_duty_allotted DATE,
    PRIMARY KEY(Bus_no,Driver_no),
    FOREIGN KEY(Bus_no) REFERENCES Bus(Bus_no),
    FOREIGN KEY(Driver_no) REFERENCES Driver(Driver_no)
);


-- Insert Records

INSERT INTO Route VALUES
(1,'Pune','Mumbai',12),
(2,'Pune','Nashik',10),
(3,'Pune','Satara',8),
(4,'Pune','Kolhapur',15),
(5,'Pune','Solapur',11),
(6,'Pune','Aurangabad',13),
(7,'Pune','Ahmednagar',9),
(8,'Pune','Lonavala',6);
```

```
INSERT INTO Bus VALUES
(10,40,'Swargate',1),
(11,50,'Shivajinagar',2),
(12,45,'Pimpri',3),
(13,40,'Hadapsar',4),
(14,55,'Swargate',5),
(15,40,'Katraj',6),
(16,48,'Pimpri',4),
(17,60,'Shivajinagar',7);


INSERT INTO Driver VALUES
(101,'Ramesh',2001,'Pune',45,35000),
(102,'Suresh',2002,'Pune',38,28000),
(103,'Mahesh',2003,'Pune',50,42000),
(104,'Amit',2004,'Pune',32,31000),
(105,'Rajesh',2005,'Pune',41,33000),
(106,'Vijay',2006,'Pune',36,27000),
(107,'Anil',2007,'Pune',48,36000),
(108,'Sunil',2008,'Pune',29,29000);

INSERT INTO Bus_Driver VALUES
(10,101,1,'2013-09-26'),
(11,102,2,'2013-09-20'),
(12,103,1,'2013-09-26'),
(13,104,2,'2013-09-15'),
```

```
(14,105,1,'2013-09-18'),
(15,106,2,'2013-09-21'),
(16,107,1,'2013-09-23'),
(17,108,2,'2013-09-25');

-- Display Driver Names
SELECT driver_name
FROM Driver;

-- Display Route Details where Bus Capacity = 40
SELECT r.*
FROM Route r
JOIN Bus b ON r.Route_no = b.Route_no
WHERE b.capacity = 40;


-- Insert 2 Records into Route Table
INSERT INTO Route VALUES
(9,'Pune','Nagpur',18),
(10,'Pune','Goa',20);


-- Display Bus Details of Route_no = 4
SELECT *
FROM Bus
WHERE Route_no = 4;
```

```
-- Q2 Create View (Display driver details whose duty on 26-09-2013)

CREATE VIEW driver_duty_26sept AS
SELECT d.*
FROM Driver d
JOIN Bus_Driver bd ON d.Driver_no = bd.Driver_no
WHERE bd.date_of_duty_allotted = '2013-09-26';

-- Run View
SELECT * FROM driver_duty_26sept;

-- Function Using Cursor (Display all duty dates for a driver)
CREATE OR REPLACE FUNCTION driver_duty_dates(dname VARCHAR)
RETURNS VOID
AS
$$
DECLARE
   rec RECORD;
   cur CURSOR FOR
   SELECT bd.date_of_duty_allotted
   FROM Bus_Driver bd
   JOIN Driver d ON bd.Driver_no = d.Driver_no
   WHERE d.driver_name = dname;
BEGIN

OPEN cur;
```

```
LOOP
FETCH cur INTO rec;
EXIT WHEN NOT FOUND;
RAISE NOTICE 'Duty Date: %', rec.date_of_duty_allotted;
END LOOP;

CLOSE cur;

END;
$$
LANGUAGE plpgsql;
-- Run Function
SELECT driver_duty_dates('Ramesh');
```

Slip 6

Q.1) Create the following database in 3NF using PostgresSQL. [Total Marks: 10]

Consider the following database of Student Competition.

Student (Sid,sname,class)

Competition (cno,cname ctype)

Relationship:

Student and Competition related with many to many relationship with attribute rank and year.

Constraints: Primary key sid,cno.

Using above database solve following Queries: [10]

1. Display students name.

2. Display name of student who took part in ‘football’.

3. List the student detail who got 1st rank.

4. List competition wise student count.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Create a View to list name of student belongs to year 2018. [10]

2. Write a Stored Function to display details of competition of ‘Pooja’. [10]

Answer:

```
-- Create Tables
-- Student Table
CREATE TABLE Student (
    sid INT PRIMARY KEY,
    sname VARCHAR(50),
    class VARCHAR(20)
);
```

```
-- Competition Table
CREATE TABLE Competition (
    cno INT PRIMARY KEY,
    cname VARCHAR(50),
    ctype VARCHAR(50)
);

-- Participation Table (Many-to-Many Relationship)
CREATE TABLE Participation (
    sid INT,
    cno INT,
    rank INT,
    year INT,
    PRIMARY KEY (sid, cno),
    FOREIGN KEY (sid) REFERENCES Student(sid),
    FOREIGN KEY (cno) REFERENCES Competition(cno)
);


-- Insert Records

-- Insert into Student
INSERT INTO Student VALUES
(1,'Pooja','FYBSc'),
(2,'Rahul','SYBSc'),
(3,'Amit','TYBSc'),
```

```
(4,'Sneha','FYBSc'),
(5,'Neha','SYBCom'),
(6,'Rohan','TYBCom'),
(7,'Kiran','FYBA'),
(8,'Anjali','SYBA');

-- Insert into Competition
INSERT INTO Competition VALUES
(101,'Intercollege Football','Football'),
(102,'Chess Championship','Indoor'),
(103,'Coding Contest','Technical'),
(104,'Debate Competition','Academic');

-- Insert into Participation
INSERT INTO Participation VALUES
(1,101,2,2018),
(2,101,1,2019),
(3,102,3,2018),
(4,103,1,2019),
(5,104,2,2018),
(6,101,4,2020),
(7,102,1,2019),
(8,103,2,2018);
```

```
-- Queries 1) Display student names
SELECT sname FROM Student;


-- 2) Display name of students who took part in Football

SELECT s.sname
FROM Student s
JOIN Participation p ON s.sid = p.sid
JOIN Competition c ON p.cno = c.cno
WHERE c.ctype = 'Football';


-- 3) List student details who got 1st rank

SELECT s.*
FROM Student s
JOIN Participation p ON s.sid = p.sid
WHERE p.rank = 1;


-- 4) Competition wise student count

SELECT c.cname, COUNT(p.sid) AS student_count
FROM Competition c
JOIN Participation p ON c.cno = p.cno
GROUP BY c.cname;
```

```
-- Q2  1) Create View for students in year 2018

CREATE VIEW Student_2018 AS
SELECT s.sname
FROM Student s
JOIN Participation p ON s.sid = p.sid
WHERE p.year = 2018;


-- Run View

SELECT * FROM Student_2018;


-- 2) Stored Function to display competition details of 'Pooja'

CREATE OR REPLACE FUNCTION get_competition_pooja()
RETURNS TABLE(
    student_name VARCHAR,
    competition_name VARCHAR,
    competition_type VARCHAR,
    rank INT,
    year INT
)
LANGUAGE plpgsql
```

```
AS
$$
BEGIN
    RETURN QUERY
    SELECT s.sname, c.cname, c.ctype, p.rank, p.year
    FROM Student s
    JOIN Participation p ON s.sid = p.sid
    JOIN Competition c ON p.cno = c.cno
    WHERE s.sname = 'Pooja';
END;
$$;


-- Run Function

SELECT * FROM get_competition_pooja();
```

Slip 7

Q.1) Create the following database in 3NF using PostgresSQL. [Total Marks: 10]

Consider a Railway Reservation System for passengers. The bogie capacity of all the bogies of

atrain is same.

Train (Train_no int, train_name varchar (20), depart_time time , arrival_time time,

source_stn varchar (20),dest_stn varchar (20), no_of_res_bogies int ,bogie_capacity int)

Passenger (Passenger_id int, passenger_name varchar (20), address varchar (30), age int, gender

char)

Relationship:

Train _Passenger: M-M relationship named ticket with descriptive attributes as follows:

Ticket (Train_no int, Passenger_id int, Ticket_no int ,bogie_no int, no_of_berths int,tdate

date, ticket_amt decimal (7, 2), ticket_status char)

Constraints: Primary key, ticket_status can be 'W' (waiting) or 'C' (confirmed).

Using above database solve following Queries:

1. Display names of ‘Shatabdi Express’ passengers.

2. Insert 2 records into ticket table.

3. List the Train Details.

4. List Ticket detail of ‘Mr.Raj’.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger after insert on passenger to display message “Age above 5 will be charged full

fare” if age of passenger is more than 5. [10]

2. Write a stored function using cursors to accept a date and find the total number of berths

reserved for all the trains on given date. [10]

Answer:

```
-- Create Tables


-- Train Table
CREATE TABLE Train(
    train_no INT PRIMARY KEY,
    train_name VARCHAR(20),
    depart_time TIME,
    arrival_time TIME,
    source_stn VARCHAR(20),
    dest_stn VARCHAR(20),
    no_of_res_bogies INT,
    bogie_capacity INT
);


-- Passenger Table
CREATE TABLE Passenger(
    passenger_id INT PRIMARY KEY,
    passenger_name VARCHAR(20),
    address VARCHAR(30),
    age INT,
    gender CHAR(1)
);
```

```
-- Ticket Table (M-M Relationship)
CREATE TABLE Ticket(
    train_no INT,
    passenger_id INT,
    ticket_no INT PRIMARY KEY,
    bogie_no INT,
    no_of_berths INT,
    tdate DATE,
    ticket_amt DECIMAL(7,2),
    ticket_status CHAR(1),

    FOREIGN KEY(train_no) REFERENCES Train(train_no),
    FOREIGN KEY(passenger_id) REFERENCES Passenger(passenger_id),

    CHECK(ticket_status IN ('W','C'))
);



-- Insert Records
-- Train Table

INSERT INTO Train VALUES
(101,'Shatabdi Express','06:00','10:00','Pune','Mumbai',10,72),
(102,'Rajdhani Express','08:00','14:00','Delhi','Mumbai',12,72),
(103,'Duronto Express','09:30','15:30','Pune','Delhi',11,72),
(104,'Garib Rath','07:00','13:00','Mumbai','Ahmedabad',10,72),
```

```
(105,'Vande Bharat','05:30','09:30','Pune','Goa',8,72),
(106,'Intercity Express','11:00','16:00','Mumbai','Pune',9,72),
(107,'Tejas Express','06:45','12:30','Mumbai','Delhi',10,72),
(108,'Jan Shatabdi','13:00','18:30','Pune','Nagpur',11,72);


--Passenger Table
INSERT INTO Passenger VALUES
(1,'Raj','Pune',25,'M'),
(2,'Pooja','Mumbai',22,'F'),
(3,'Amit','Delhi',30,'M'),
(4,'Sneha','Pune',18,'F'),
(5,'Rohan','Mumbai',35,'M'),
(6,'Anjali','Delhi',27,'F'),
(7,'Kiran','Pune',10,'M'),
(8,'Neha','Mumbai',28,'F');

-- Ticket Table
INSERT INTO Ticket VALUES
(101,1,1001,1,2,'2024-03-01',1500.00,'C'),
(101,2,1002,2,1,'2024-03-01',1500.00,'C'),
(102,3,1003,3,2,'2024-03-02',2000.00,'W'),
(103,4,1004,4,1,'2024-03-02',1800.00,'C'),
(104,5,1005,2,2,'2024-03-03',1200.00,'C'),
(101,6,1006,5,1,'2024-03-03',1500.00,'W'),
(102,7,1007,6,1,'2024-03-04',2000.00,'C'),
(104,8,1008,3,2,'2024-03-04',1200.00,'W');
```

```
-- Queries 1) Display names of Shatabdi Express passengers

SELECT p.passenger_name
FROM Passenger p
JOIN Ticket t ON p.passenger_id = t.passenger_id
JOIN Train tr ON t.train_no = tr.train_no
WHERE tr.train_name = 'Shatabdi Express';


-- 2) Insert 2 Records into Ticket Table

INSERT INTO Ticket VALUES
(103,1,1009,3,1,'2024-03-05',1800.00,'C'),
(105,2,1010,4,2,'2024-03-05',2200.00,'W');


-- 3) List Train Details
SELECT * FROM Train;


-- 4) List Ticket Details of Mr. Raj
SELECT t.*
FROM Ticket t
JOIN Passenger p ON t.passenger_id = p.passenger_id
WHERE p.passenger_name = 'Raj';
```

```
-- Q2  1) Trigger

-- Trigger Function

CREATE OR REPLACE FUNCTION check_age()
RETURNS TRIGGER
LANGUAGE plpgsql
AS
$$
BEGIN
   IF NEW.age > 5 THEN
      RAISE NOTICE 'Age above 5 will be charged full fare';
   END IF;
   RETURN NEW;
END;
$$;

-- Create Trigger

CREATE TRIGGER passenger_trigger
AFTER INSERT ON Passenger
FOR EACH ROW
EXECUTE FUNCTION check_age();
```

-- 2) Stored Function Using Cursor

-- Find Total Berths Reserved on Given Date

```
CREATE OR REPLACE FUNCTION total_berths_reserved(input_date DATE)
RETURNS INT
LANGUAGE plpgsql
AS
$$
DECLARE
    total INT := 0;
    berth INT;

    cur_ticket CURSOR FOR
        SELECT no_of_berths
        FROM Ticket
        WHERE tdate = input_date;

BEGIN
    OPEN cur_ticket;

    LOOP
        FETCH cur_ticket INTO berth;
        EXIT WHEN NOT FOUND;
        total := total + berth;
    END LOOP;
```

```
    CLOSE cur_ticket;

    RETURN total;
END;
$$;


-- Run Function

SELECT total_berths_reserved('2024-03-01');
```

Slip 8

Q.1) Create the following database in 3NF using PostgresSQL. [Total Marks: 10]

Consider a Railway Reservation System for passengers. The bogie capacity of all the bogies of

atrain is same.

Train (Train_no int, train_name varchar (20), depart_time time , arrival_time time,

source_stn varchar (20),dest_stn varchar (20), no_of_res_bogies int ,bogie_capacity int)

Passenger (Passenger_id int, passenger_name varchar (20), address varchar (30), age int, gender

char)

Relationship:

Train _Passenger: M-M relationship named ticket with descriptive attributes as follows:

Ticket (Train_no int, Passenger_id int, Ticket_no int, bogie_no int, no_of_berths int,tdate

date, ticket_amt decimal (7, 2), ticket_status char)

Constraints: Primary key, ticket_status can be 'W' (waiting) or 'C' (confirmed).

Using above database solve following Queries:

1. Display names of ‘Koyana Express’ passengers.

2. Display count of confirmed bookings of ‘Rajdhani Express’

3. Sort the Passenger details as per age.

4. Display name of passenger whose age<5.

.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Create a view to display the passenger details whose status is confirmed.

[10]

2.Write a stored function to display train wise bookings on 12-04-2021 whose ticket status is

waiting. [10]

Answer:

```
-- Create Tables


-- Train Table
CREATE TABLE Train(
    train_no INT PRIMARY KEY,
    train_name VARCHAR(20),
    depart_time TIME,
    arrival_time TIME,
    source_stn VARCHAR(20),
    dest_stn VARCHAR(20),
    no_of_res_bogies INT,
    bogie_capacity INT
);


-- Passenger Table
CREATE TABLE Passenger(
    passenger_id INT PRIMARY KEY,
    passenger_name VARCHAR(20),
    address VARCHAR(30),
    age INT,
    gender CHAR(1)
);
```

```
-- Ticket Table (M-M relationship)
CREATE TABLE Ticket(
    train_no INT,
    passenger_id INT,
    ticket_no INT PRIMARY KEY,
    bogie_no INT,
    no_of_berths INT,
    tdate DATE,
    ticket_amt DECIMAL(7,2),
    ticket_status CHAR(1),

    FOREIGN KEY(train_no) REFERENCES Train(train_no),
    FOREIGN KEY(passenger_id) REFERENCES Passenger(passenger_id),

    CHECK(ticket_status IN ('W','C'))
);



-- Insert Records

-- Train Table

INSERT INTO Train VALUES
(101,'Koyana Express','06:00','11:00','Pune','Kolhapur',10,72),
(102,'Rajdhani Express','08:00','14:00','Delhi','Mumbai',12,72),
(103,'Duronto Express','09:30','15:30','Pune','Delhi',11,72),
```

(104,'Garib Rath','07:00','13:00','Mumbai','Ahmedabad',10,72),
(105,'Shatabdi Express','05:30','09:30','Pune','Mumbai',8,72),
(106,'Intercity Express','11:00','16:00','Mumbai','Pune',9,72),
(107,'Tejas Express','06:45','12:30','Mumbai','Delhi',10,72),
(108,'Jan Shatabdi','13:00','18:30','Pune','Nagpur',11,72);

-- Passenger Table

INSERT INTO Passenger VALUES
(1,'Raj','Pune',25,'M'),
(2,'Pooja','Mumbai',22,'F'),
(3,'Amit','Delhi',30,'M'),
(4,'Sneha','Pune',18,'F'),
(5,'Rohan','Mumbai',35,'M'),
(6,'Anjali','Delhi',27,'F'),
(7,'Kiran','Pune',4,'M'),
(8,'Neha','Mumbai',3,'F');

-- Ticket Table

INSERT INTO Ticket VALUES
(101,1,1001,1,2,'2021-04-12',1500.00,'C'),
(101,2,1002,2,1,'2021-04-12',1500.00,'W'),
(102,3,1003,3,2,'2021-04-12',2000.00,'C'),
(102,4,1004,4,1,'2021-04-12',2000.00,'C'),
(103,5,1005,2,2,'2021-04-12',1800.00,'W'),
(104,6,1006,5,1,'2021-04-12',1200.00,'C'),

```
(101,7,1007,6,1,'2021-04-12',1500.00,'W'),
(102,8,1008,3,2,'2021-04-12',2000.00,'W');

 -- Queries  1) Display names of Koyana Express passengers

SELECT p.passenger_name
FROM Passenger p
JOIN Ticket t ON p.passenger_id = t.passenger_id
JOIN Train tr ON t.train_no = tr.train_no
WHERE tr.train_name = 'Koyana Express';

-- 2) Display count of confirmed bookings of Rajdhani Express

SELECT COUNT(*) AS confirmed_bookings
FROM Ticket t
JOIN Train tr ON t.train_no = tr.train_no
WHERE tr.train_name = 'Rajdhani Express'
AND t.ticket_status = 'C';

-- 3) Sort passenger details as per age

SELECT * FROM Passenger
ORDER BY age;

-- 4) Display name of passenger whose age < 5

SELECT passenger_name
```

```
FROM Passenger
WHERE age < 5;


-- Q2 1) Create View for confirmed passengers

CREATE VIEW Confirmed_Passengers AS
SELECT p.passenger_id,
    p.passenger_name,
    p.address,
    p.age,
    p.gender,
    t.ticket_status
FROM Passenger p
JOIN Ticket t ON p.passenger_id = t.passenger_id
WHERE t.ticket_status = 'C';


-- Run View

SELECT * FROM Confirmed_Passengers;

-- 2) Stored Function

-- Display train wise bookings on 12-04-2021 whose ticket status is waiting
```

```
CREATE OR REPLACE FUNCTION waiting_bookings()
RETURNS TABLE(
   train_name VARCHAR,
   total_waiting INT
)
LANGUAGE plpgsql
AS
$$
BEGIN
RETURN QUERY
SELECT tr.train_name,
      COUNT(t.ticket_no)
FROM Train tr
JOIN Ticket t ON tr.train_no = t.train_no
WHERE t.ticket_status = 'W'
AND t.tdate = '2021-04-12'
GROUP BY tr.train_name;
END;
$$;



-- Run Function

SELECT * FROM waiting_bookings();
```

Slip 9

Q.1) Create the following database in 3NF using PostgresSQL. [Total Marks: 10]

Consider the following Project-Employee database, which is managed by a company and stores

the details of projects assigned to employees.

Project (Pno int, pname varchar (30), ptype varchar (20), duration integer)

Employee (Eno integer, ename varchar (20), qualification char (15), joining_date date)

Relationship:

Project-Employee related with many-to-many relationship, with descriptive attributes as

start_date_of_Project, no_of_hours_worked.

Constraints: Primary key, pname should not be null.

Using above database solve following Queries:

1. Display the project name, project type and project start date.

2. Display details of employees working on ‘AI’ project.

3. List name of employee whose qualification is ‘M.Sc’.

4. Count the number of employee.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger before inserting the duration into the project table and make sure that the

duration is always greater than zero. Display appropriate message. [10]

2. Write function to accept project name as an input parameter and display names of

employees working on that project. [10]

Answer:

```
-- Create Tables

-- Project Table
CREATE TABLE Project(
    pno INT PRIMARY KEY,
    pname VARCHAR(30) NOT NULL,
    ptype VARCHAR(20),
    duration INT
);

-- Employee Table
CREATE TABLE Employee(
    eno INT PRIMARY KEY,
    ename VARCHAR(20),
    qualification VARCHAR(15),
    joining_date DATE
);

-- Relationship Table (Many-to-Many)
CREATE TABLE Project_Employee(
    pno INT,
    eno INT,
    start_date_of_project DATE,
    no_of_hours_worked INT,

    PRIMARY KEY(pno, eno),
```

```
    FOREIGN KEY(pno) REFERENCES Project(pno),
    FOREIGN KEY(eno) REFERENCES Employee(eno)
);


-- Insert Records

-- Project Table

INSERT INTO Project VALUES
(101,'AI','Research',12),
(102,'Web Development','Software',8),
(103,'Data Mining','Research',10),
(104,'Cloud Migration','Infrastructure',6),
(105,'Cyber Security','Security',9),
(106,'Mobile App','Software',7),
(107,'IoT Automation','Hardware',11),
(108,'Blockchain','Research',10);


--Employee Table

INSERT INTO Employee VALUES
(1,'Raj','M.Sc','2019-05-10'),
(2,'Pooja','B.Tech','2020-06-15'),
(3,'Amit','M.Sc','2018-03-12'),
```

(4,'Sneha','MCA','2021-07-01'),

(5,'Rohan','B.Sc','2017-02-20'),

(6,'Anjali','M.Sc','2019-09-18'),

(7,'Kiran','B.Tech','2022-01-11'),

(8,'Neha','MBA','2020-12-05');

-- Project_Employee Table

INSERT INTO Project_Employee VALUES

(101,1,'2023-01-10',120),

(101,3,'2023-01-15',100),

(102,2,'2023-02-01',90),

(103,4,'2023-03-10',80),

(104,5,'2023-04-05',110),

(105,6,'2023-05-12',95),

(106,7,'2023-06-20',70),

(101,8,'2023-07-01',85);

-- Queries 1) Display project name, project type and project start date

SELECT p.pname, p.ptype, pe.start_date_of_project

FROM Project p

JOIN Project_Employee pe ON p.pno = pe.pno;

```
-- 2) Display details of employees working on AI project

SELECT e.*
FROM Employee e
JOIN Project_Employee pe ON e.eno = pe.eno
JOIN Project p ON pe.pno = p.pno
WHERE p.pname = 'AI';


-- 3) List name of employee whose qualification is M.Sc

SELECT ename
FROM Employee
WHERE qualification = 'M.Sc';


-- 4) Count number of employees

SELECT COUNT(*) AS total_employees
FROM Employee;


-- Q2  1) Trigger

-- Ensure duration > 0 before inserting into Project

-- Trigger Function
```

```
CREATE OR REPLACE FUNCTION check_duration()
RETURNS TRIGGER
LANGUAGE plpgsql
AS
$$
BEGIN
   IF NEW.duration <= 0 THEN
      RAISE EXCEPTION 'Duration must be greater than zero';
   END IF;
   RETURN NEW;
END;
$$;



-- Create Trigger

CREATE TRIGGER duration_trigger
BEFORE INSERT ON Project
FOR EACH ROW
EXECUTE FUNCTION check_duration();

-- 2) Stored Function
-- Accept project name and display employees working on that project.

CREATE OR REPLACE FUNCTION employees_on_project(p_project_name
VARCHAR)
RETURNS TABLE(employee_name VARCHAR)
```

```
LANGUAGE plpgsql
AS
$$
BEGIN
RETURN QUERY
SELECT e.ename
FROM Employee e
JOIN Project_Employee pe ON e.eno = pe.eno
JOIN Project p ON pe.pno = p.pno
WHERE p.pname = p_project_name;
END;
$$;


-- Run Function

SELECT * FROM employees_on_project('AI');
```

Slip 10

Q.1) Create the following database in 3NF using PostgresSQL [Total Marks: 10]

Consider the following Project-Employee database, which is managed by a company and

stores the details of projects assigned to employees.

Project (Pno int, pname varchar (30), ptype varchar (20), duration integer)

Employee (Eno integer, ename varchar (20), qualification char (15), joining_date date)

Relationship:

Project-Employee related with many-to-many relationship, with descriptive attributes as

start_date_of_Project, no_of_hours_worked.

Constraints: Primary key, pname should not be null.

Using above database solve following Queries:

1. List the name of employee whose name end by letter ‘L’.

2. Insert 2 records into employee table.

3. Display Project details which start on date’ 20-9-2017’.

4. Display Project details whose duration>3.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger before deleting an employee record from employee table. Raise a

notice and display the message “Employee record is being deleted”. [10]

2.Create a View To display employee details and it should be sorted by employee’s

name in descending order [10]

Answer:

```
-- Create Tables

-- Project Table
CREATE TABLE Project(
    pno INT PRIMARY KEY,
    pname VARCHAR(30) NOT NULL,
    ptype VARCHAR(20),
    duration INT
);

-- Employee Table
CREATE TABLE Employee(
    eno INT PRIMARY KEY,
    ename VARCHAR(20),
    qualification VARCHAR(15),
    joining_date DATE
);

-- Relationship Table (Many-to-Many)
CREATE TABLE Project_Employee(
    pno INT,
    eno INT,
    start_date_of_project DATE,
    no_of_hours_worked INT,
```

```
    PRIMARY KEY(pno, eno),

    FOREIGN KEY(pno) REFERENCES Project(pno),
    FOREIGN KEY(eno) REFERENCES Employee(eno)
);



-- Insert Records
-- Project Table

INSERT INTO Project VALUES
(101,'AI System','Research',5),
(102,'Web Portal','Software',4),
(103,'Data Mining','Research',6),
(104,'Cloud System','Infrastructure',2),
(105,'Security Tool','Security',7),
(106,'Mobile App','Software',3),
(107,'IoT System','Hardware',8),
(108,'Blockchain App','Research',5);

-- Employee Table
INSERT INTO Employee VALUES
(1,'Rahul','M.Sc','2017-03-10'),
(2,'Pooja','B.Tech','2018-05-12'),
(3,'Anil','MCA','2016-06-14'),
(4,'Sunil','B.Sc','2017-07-18'),
(5,'Nehal','MBA','2019-02-22'),
```

```
(6,'Kunal','M.Sc','2020-01-11'),
(7,'Amit','B.Tech','2021-09-01'),
(8,'Komal','MCA','2017-08-15');


-- Project_Employee Table
INSERT INTO Project_Employee VALUES
(101,1,'2017-09-20',120),
(102,2,'2017-08-15',90),
(103,3,'2017-09-20',100),
(104,4,'2018-01-10',70),
(105,5,'2019-02-12',110),
(106,6,'2017-09-20',95),
(107,7,'2020-03-15',80),
(108,8,'2021-04-20',85);



-- Queries   1) List employees whose name ends with 'L'
SELECT ename
FROM Employee
WHERE ename LIKE '%l' OR ename LIKE '%L';

-- 2) Insert 2 records into Employee table
INSERT INTO Employee VALUES
(9,'Rohit','B.Sc','2022-03-12'),
(10,'Nikhil','M.Sc','2023-01-10');
```

```
-- 3) Display project details which start on 20-09-2017

SELECT p.*
FROM Project p
JOIN Project_Employee pe ON p.pno = pe.pno
WHERE pe.start_date_of_project = '2017-09-20';

-- 4) Display project details whose duration > 3

SELECT *
FROM Project
WHERE duration > 3;

-- Q2  1) Trigger Before Deleting Employee

-- Trigger Function

CREATE OR REPLACE FUNCTION delete_employee_notice()
RETURNS TRIGGER
LANGUAGE plpgsql
AS
$$
BEGIN
   RAISE NOTICE 'Employee record is being deleted';
   RETURN OLD;
END;
$$;
```

```
-- Create Trigger

CREATE TRIGGER employee_delete_trigger
BEFORE DELETE ON Employee
FOR EACH ROW
EXECUTE FUNCTION delete_employee_notice();

-- 2) Create View (Employees sorted by name DESC)

CREATE VIEW Employee_Sorted_View AS
SELECT *
FROM Employee
ORDER BY ename DESC;

-- Run View

SELECT * FROM Employee_Sorted_View;
```

Slip 11

Q.1) Practical Questions on PostgresSQL [Total Marks 10]

Consider the following database

Student (sname, rollno, phone)

Course (cname, ccode, duration)

A student can enroll in one or more courses. A course can have many students → Many-to-Many

relationship.

Using above database, solve the following queries:

1. List details of courses with duration ‘6 months’

2. List student-wise course details.

3. Delete the student record of ‘Rohan’.

4. Update phone number of ‘Riya’ to 987654321.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger before insert record of Student. If the rollno is less than or equal to zero and

sname is null then give the appropriate message [10]

2. Write a stored function to accept course code as an input parameter and display course name with duration. [10]

Answer :

```
-- Create Student table
CREATE TABLE Student (
    rollno INT PRIMARY KEY,
    sname VARCHAR(50),
    phone BIGINT
);
```

```
-- Create Course table
CREATE TABLE Course (
    ccode VARCHAR(10) PRIMARY KEY,
    cname VARCHAR(50),
    duration VARCHAR(20)
);

-- Junction table for Many-to-Many relationship
CREATE TABLE Enrollment (
    rollno INT,
    ccode VARCHAR(10),
    PRIMARY KEY (rollno, ccode),
    FOREIGN KEY (rollno) REFERENCES Student(rollno) ON DELETE CASCADE,
    FOREIGN KEY (ccode) REFERENCES Course(ccode) ON DELETE CASCADE
);

-- Insert Students
INSERT INTO Student VALUES
(1, 'Rohan', 9123456780),
(2, 'Riya', 9988776655),
(3, 'Amit', 9876501234),
(4, 'Sneha', 9765432109);

-- Insert Courses
INSERT INTO Course VALUES
```

```
('C101', 'Database', '6 months'),
('C102', 'Python', '3 months'),
('C103', 'Java', '6 months'),
('C104', 'Web Dev', '4 months');

-- Insert Enrollments
INSERT INTO Enrollment VALUES
(1, 'C101'),
(1, 'C102'),
(2, 'C101'),
(3, 'C103'),
(4, 'C104'),
(2, 'C103'),
(3, 'C102'),
(4, 'C101');

-----------------------------------------------
-- Q1 Answers
-----------------------------------------------

-- 1. Courses with duration ‘6 months’
SELECT *
FROM Course
WHERE duration = '6 months';

-- 2. Student-wise course details
SELECT s.sname, s.rollno, c.cname, c.duration
```

```
FROM Student s
JOIN Enrollment e ON s.rollno = e.rollno
JOIN Course c ON e.ccode = c.ccode
ORDER BY s.sname;

-- 3. Delete student ‘Rohan’
DELETE FROM Student
WHERE sname = 'Rohan';

-- 4. Update phone number of ‘Riya’
UPDATE Student
SET phone = 987654321
WHERE sname = 'Riya';

-----------------------------------------------
-- Q2 Answers
-----------------------------------------------

-- Trigger Function
CREATE OR REPLACE FUNCTION check_student()
RETURNS TRIGGER AS $$
BEGIN
   IF NEW.rollno <= 0 OR NEW.sname IS NULL THEN
      RAISE EXCEPTION 'Invalid data: rollno must be > 0 and name cannot be
NULL';
   END IF;
   RETURN NEW;
END;
```

```
$$ LANGUAGE plpgsql;

-- Create Trigger
CREATE TRIGGER student_trigger
BEFORE INSERT ON Student
FOR EACH ROW
EXECUTE FUNCTION check_student();

-----------------------------------------------
-- Stored Function
-----------------------------------------------

CREATE OR REPLACE FUNCTION get_course_details(p_ccode VARCHAR)
RETURNS TABLE(cname VARCHAR, duration VARCHAR) AS $$
BEGIN
   RETURN QUERY
   SELECT c.cname, c.duration
   FROM Course c
   WHERE c.ccode = p_ccode;
END;
$$ LANGUAGE plpgsql;

-- Call Function
SELECT * FROM get_course_details('C101');
```

Slip 12

Q.1) Practical Questions on PostgresSQL [Total Marks 10]

Consider the following database

Employee (emp_id, emp_name, address, bdate)

Investor (inv_no, inv_name, inv_date,inv_amt)

An employee may invest in one or more investments; hence he can be an investor. But an

investor need not be an employee of the firm.

Assume appropriate data types for all the attributes.

Using above database, solve the following queries:

1. List the names of employees.

2. List the information of Investors whose name start with ‘M’

3. Increase the amount of investors by 10%.

4. Delete the investor information whose amount is 20,000.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger to validate the investment amount approved. It must be less than or

equal to 2000/-. Display appropriate message. [10]

2. Write a stored function to count number of investor. (Accept investor name as an input

parameter). Display message for invalid investor name. [10]

Answer :

-- CREATE TABLES

```
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    address VARCHAR(100),
    bdate DATE
);

CREATE TABLE Investor (
    inv_no INT PRIMARY KEY,
    inv_name VARCHAR(50),
    inv_date DATE,
    inv_amt NUMERIC(10,2)
);

-----------------------------------------------
-- INSERT DATA
-----------------------------------------------

INSERT INTO Employee VALUES
(1, 'Rohan', 'Pune', '1995-05-10'),
(2, 'Meena', 'Mumbai', '1992-08-15'),
(3, 'Amit', 'Delhi', '1990-02-20'),
(4, 'Sneha', 'Nashik', '1998-11-25');

INSERT INTO Investor VALUES
(101, 'Mohan', '2023-01-10', 1500),
(102, 'Riya', '2023-02-15', 20000),
```

```
(103, 'Manish', '2023-03-20', 1800),
(104, 'Suresh', '2023-04-25', 1200);

-----------------------------------------------
-- Q1 QUERIES
-----------------------------------------------

-- 1. List names of employees
SELECT emp_name FROM Employee;

-- 2. Investors whose name starts with 'M'
SELECT *
FROM Investor
WHERE inv_name LIKE 'M%';

-- 3. Increase investor amount by 10%
UPDATE Investor
SET inv_amt = inv_amt * 1.10;

-- 4. Delete investor whose amount is 20000
DELETE FROM Investor
WHERE inv_amt = 20000;

-----------------------------------------------
-- Q2: TRIGGER (Validate Investment <= 2000)
-----------------------------------------------
```

```
CREATE OR REPLACE FUNCTION validate_investment()
RETURNS TRIGGER AS $$
BEGIN
   IF NEW.inv_amt > 2000 THEN
      RAISE EXCEPTION 'Investment amount must be ≤ 2000';
   END IF;

   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger
DROP TRIGGER IF EXISTS investment_trigger ON Investor;

CREATE TRIGGER investment_trigger
BEFORE INSERT ON Investor
FOR EACH ROW
EXECUTE FUNCTION validate_investment();

-----------------------------------------------
-- Q2: STORED FUNCTION
-- Count investors by name
-----------------------------------------------

CREATE OR REPLACE FUNCTION count_investor(p_name VARCHAR)
RETURNS TEXT AS $$
DECLARE
```

```
    total INT;
BEGIN
    SELECT COUNT(*) INTO total
    FROM Investor
    WHERE inv_name = p_name;

    IF total = 0 THEN
        RETURN 'Invalid investor name';
    END IF;

    RETURN 'Number of investors: ' || total;
END;
$$ LANGUAGE plpgsql;

-----------------------------------------------
-- FUNCTION CALL
-----------------------------------------------

SELECT count_investor('Mohan');
```

Slip 13

Q.1) Practical Questions on PostgresSQL [Total Marks 10]

Consider the following database

Employee: (ename, empid, phone)

Department: (dname, deptid, location)

Relationship: An employee belongs to exactly one department; a department has many employees.

Using above database, solve the following queries:

1. List employees in 'Computer science' department

2. Delete the employee whose names starts with 'S'

3. Insert 2 records into Deapartment.

4. Display employee name along with department.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Create a View:

1. To display Employee details that have birth date before 1900 year. [10]

2. To display employee name with phone number.

2. Write a cursor to display employee details along with their department name. [10]

Answer:

```
-----------------------------------------------
-- CREATE TABLES
-----------------------------------------------

CREATE TABLE Department (
    deptid INT PRIMARY KEY,
```

```
    dname VARCHAR(50),
    location VARCHAR(50)
);

CREATE TABLE Employee (
    empid INT PRIMARY KEY,
    ename VARCHAR(50),
    phone VARCHAR(15),
    birthdate DATE,
    deptid INT,
    FOREIGN KEY (deptid) REFERENCES Department(deptid)
);

-----------------------------------------------
-- INSERT DATA INTO DEPARTMENT
-----------------------------------------------

INSERT INTO Department VALUES
(1, 'Computer Science', 'Pune'),
(2, 'Mechanical', 'Mumbai'),
(3, 'Electrical', 'Delhi'),
(4, 'Civil', 'Chennai');

-----------------------------------------------
-- INSERT DATA INTO EMPLOYEE
-----------------------------------------------
```

```
INSERT INTO Employee VALUES
(101, 'Amit', '9876543210', '1985-06-15', 1),
(102, 'Sneha', '9123456780', '1990-03-22', 1),
(103, 'Suresh', '9988776655', '1975-01-10', 2),
(104, 'Ravi', '9090909090', '1885-07-19', 3),
(105, 'Sunita', '8888888888', '1895-11-11', 1),
(106, 'Kiran', '7777777777', '2000-05-25', 4),
(107, 'Sanjay', '6666666666', '1988-09-09', 2),
(108, 'Neha', '9999999999', '1995-12-12', 1);

-----------------------------------------------
-- LIST EMPLOYEES IN COMPUTER SCIENCE
-----------------------------------------------

SELECT e.*
FROM Employee e
JOIN Department d ON e.deptid = d.deptid
WHERE d.dname = 'Computer Science';

-----------------------------------------------
-- DELETE EMPLOYEES STARTING WITH 'S'
-----------------------------------------------

DELETE FROM Employee
WHERE ename LIKE 'S%';

-----------------------------------------------
```

```
-- INSERT 2 RECORDS INTO DEPARTMENT
-----------------------------------------------

INSERT INTO Department VALUES
(5, 'Information Technology', 'Bangalore'),
(6, 'Electronics', 'Hyderabad');

-----------------------------------------------
-- DISPLAY EMPLOYEE WITH DEPARTMENT
-----------------------------------------------

SELECT e.ename, d.dname
FROM Employee e
JOIN Department d ON e.deptid = d.deptid;

-----------------------------------------------
-- VIEW 1: Employees born before 1900
-----------------------------------------------

CREATE OR REPLACE VIEW Old_Employees AS
SELECT *
FROM Employee
WHERE birthdate < DATE '1900-01-01';

-----------------------------------------------
-- VIEW 2: Employee name and phone
-----------------------------------------------
```

```
CREATE OR REPLACE VIEW Emp_Phone_View AS
SELECT ename, phone
FROM Employee;

-----------------------------------------------
-- CURSOR: Employee + Department details
-----------------------------------------------

DO $$
DECLARE
   emp_record RECORD;

   emp_cursor CURSOR FOR
      SELECT e.ename, e.phone, d.dname
      FROM Employee e
      JOIN Department d ON e.deptid = d.deptid;

BEGIN
   OPEN emp_cursor;

   LOOP
      FETCH emp_cursor INTO emp_record;
      EXIT WHEN NOT FOUND;

      RAISE NOTICE 'Name: %, Phone: %, Department: %',
         emp_record.ename,
```

```
        emp_record.phone,
        emp_record.dname;
    END LOOP;

    CLOSE emp_cursor;
END $$;
```

Slip 14

Q.1) Practical Questions on PostgresSQL [Total Marks 10]

Consider the following database

Movie (M_Name,release_year,budget)

Actor (A_name, role,charges,a_address)

Producer (producer_id, Name, P_address)

Each actor has acted in one or more movies. Each producer has produced many movies and each movie can be produced by more than one producers. Each movie has one or more actors acting in it, in different roles.

Assume appropriate data types for all the attributes.

Using above database, solve the following queries:

1. Display the maximum budget.

2. List the names of producers of “Chhava” movie.

3. List the names of actors who live in Pune.

4. Insert 2 records into producers table.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger before inserting the actor record in Actor table, if the age of actor is between 0 and 3, then display error message ‘Invalid input’. [10]

2. Write a stored function to display details of movies who release before year 2000 .[10]

Answer:

```
-----------------------------------------------

-- CREATE TABLES

-----------------------------------------------


CREATE TABLE Movie (
```

```
    m_name VARCHAR(50) PRIMARY KEY,
    release_year INT,
    budget NUMERIC
);

CREATE TABLE Producer (
    producer_id INT PRIMARY KEY,
    name VARCHAR(50),
    p_address VARCHAR(100)
);

CREATE TABLE Actor (
    a_name VARCHAR(50),
    role VARCHAR(50),
    charges NUMERIC,
    a_address VARCHAR(100),
    age INT,
    m_name VARCHAR(50),
    FOREIGN KEY (m_name) REFERENCES Movie(m_name)
);

-- Many-to-Many relationship
CREATE TABLE Movie_Producer (
    m_name VARCHAR(50),
    producer_id INT,
    PRIMARY KEY (m_name, producer_id),
    FOREIGN KEY (m_name) REFERENCES Movie(m_name) ON DELETE
CASCADE,
```

```
    FOREIGN KEY (producer_id) REFERENCES Producer(producer_id) ON
DELETE CASCADE
);

-----------------------------------------------
-- INSERT DATA INTO MOVIE
-----------------------------------------------

INSERT INTO Movie VALUES
('Chhava', 2023, 150000000),
('War', 2019, 200000000),
('Dangal', 2016, 100000000),
('Lagaan', 2001, 25000000),
('Sholay', 1975, 30000000),
('Koi Mil Gaya', 2003, 35000000),
('Krish', 2006, 40000000),
('Border', 1997, 20000000);

-----------------------------------------------
-- INSERT PRODUCERS
-----------------------------------------------

INSERT INTO Producer VALUES
(1, 'Karan Johar', 'Mumbai'),
(2, 'Aditya Chopra', 'Mumbai'),
(3, 'Ritesh Sidhwani', 'Pune'),
(4, 'Farhan Akhtar', 'Delhi'),
(5, 'Ashutosh Gowariker', 'Mumbai'),
```

```
(6, 'Sajid Nadiadwala', 'Pune'),
(7, 'Boney Kapoor', 'Chennai'),
(8, 'Subhash Ghai', 'Mumbai');

-----------------------------------------------
-- MOVIE-PRODUCER RELATION
-----------------------------------------------

INSERT INTO Movie_Producer VALUES
('Chhava', 1),
('Chhava', 2),
('War', 2),
('Dangal', 5),
('Lagaan', 5),
('Sholay', 7),
('Border', 8),
('Krish', 6);

-----------------------------------------------
-- INSERT ACTORS
-----------------------------------------------

INSERT INTO Actor VALUES
('Aamir Khan', 'Lead', 5000000, 'Mumbai', 55, 'Dangal'),
('Hrithik Roshan', 'Lead', 7000000, 'Pune', 50, 'War'),
('Salman Khan', 'Lead', 8000000, 'Mumbai', 57, 'Chhava'),
('Shah Rukh Khan', 'Lead', 9000000, 'Delhi', 58, 'Chhava'),
```

```
('Amitabh Bachchan', 'Supporting', 6000000, 'Pune', 80, 'Sholay'),
('Sunny Deol', 'Lead', 4000000, 'Pune', 65, 'Border'),
('Preity Zinta', 'Lead', 3000000, 'Shimla', 48, 'Koi Mil Gaya'),
('Kangana Ranaut', 'Lead', 3500000, 'Pune', 36, 'Krish');

-----------------------------------------------
-- Q1.1 MAX BUDGET
-----------------------------------------------

SELECT MAX(budget) AS max_budget
FROM Movie;

-----------------------------------------------
-- Q1.2 PRODUCERS OF CHHAVA
-----------------------------------------------

SELECT p.name
FROM Producer p
JOIN Movie_Producer mp ON p.producer_id = mp.producer_id
WHERE mp.m_name = 'Chhava';

-----------------------------------------------
-- Q1.3 ACTORS LIVING IN PUNE
-----------------------------------------------

SELECT a_name
FROM Actor
```

```
WHERE a_address = 'Pune';

-----------------------------------------------
-- Q1.4 INSERT 2 PRODUCERS
-----------------------------------------------

INSERT INTO Producer VALUES
(9, 'Ekta Kapoor', 'Mumbai'),
(10, 'Rajkumar Hirani', 'Pune');

-----------------------------------------------
-- TRIGGER: AGE VALIDATION
-----------------------------------------------

CREATE OR REPLACE FUNCTION check_actor_age()
RETURNS TRIGGER AS $$
BEGIN
   IF NEW.age BETWEEN 0 AND 3 THEN
      RAISE EXCEPTION 'Invalid input';
   END IF;

   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS actor_age_trigger ON Actor;
```

```
CREATE TRIGGER actor_age_trigger
BEFORE INSERT ON Actor
FOR EACH ROW
EXECUTE FUNCTION check_actor_age();
-----------------------------------------------
-- STORED FUNCTION: MOVIES BEFORE 2000
-----------------------------------------------
CREATE OR REPLACE FUNCTION get_old_movies()
RETURNS TABLE (
   m_name VARCHAR,
   release_year INT,
   budget NUMERIC
) AS $$
BEGIN
   RETURN QUERY
   SELECT m.m_name, m.release_year, m.budget
   FROM Movie m
   WHERE m.release_year < 2000;
END;
$$ LANGUAGE plpgsql;

-----------------------------------------------
-- CALL FUNCTION
-----------------------------------------------

SELECT * FROM get_old_movies();
```

Slip 15

Q1) Practical Questions on PostgresSQL [Total Marks 10]

Consider the following database

Book: (btitle, ISBN,category)

Author: (aname, address, phone)

Relationship: A book has exactly one author; an author can write many books.

Using above database, solve the following queries:

1. Insert 2 records into book table.

2. List author-wise book details.

3. List books written by 'J.K. Rowling'.

4. Update phone number of 'George Martin' to 987654321

Q2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger before inserting the Book record in Book table, if Author name is not

exist then print "Invalid Name" [10]

2. Write a function using cursor to display all Book of Specific Author. [10]

Answer:

```
-----------------------------------------------
-- CREATE TABLES
-----------------------------------------------

CREATE TABLE Author (
    aname VARCHAR(100) PRIMARY KEY,
    address TEXT,
    phone VARCHAR(15)
);
```

```
CREATE TABLE Book (
    btitle VARCHAR(100),
    ISBN VARCHAR(20) PRIMARY KEY,
    category VARCHAR(50),
    aname VARCHAR(100),
    FOREIGN KEY (aname) REFERENCES Author(aname)
);

-----------------------------------------------
-- INSERT DATA INTO AUTHOR
-----------------------------------------------

INSERT INTO Author VALUES
('J.K. Rowling', 'UK', '1111111111'),
('George Martin', 'USA', '2222222222'),
('Chetan Bhagat', 'India', '3333333333'),
('Dan Brown', 'USA', '4444444444');

-----------------------------------------------
-- INSERT DATA INTO BOOK
-----------------------------------------------

INSERT INTO Book VALUES
('Harry Potter 1', 'ISBN001', 'Fantasy', 'J.K. Rowling'),
('Harry Potter 2', 'ISBN002', 'Fantasy', 'J.K. Rowling'),
('Game of Thrones', 'ISBN003', 'Fantasy', 'George Martin'),
```

```
('Clash of Kings', 'ISBN004', 'Fantasy', 'George Martin'),
('Five Point Someone', 'ISBN005', 'Fiction', 'Chetan Bhagat'),
('2 States', 'ISBN006', 'Romance', 'Chetan Bhagat'),
('Da Vinci Code', 'ISBN007', 'Thriller', 'Dan Brown'),
('Inferno', 'ISBN008', 'Thriller', 'Dan Brown');

-----------------------------------------------
-- Q1.1 INSERT 2 BOOK RECORDS
-----------------------------------------------

INSERT INTO Book VALUES
('New Book 1', 'ISBN009', 'Drama', 'Dan Brown'),
('New Book 2', 'ISBN010', 'Fantasy', 'J.K. Rowling');

-----------------------------------------------
-- Q1.2 AUTHOR-WISE BOOK DETAILS
-----------------------------------------------

SELECT A.aname, B.btitle, B.ISBN, B.category
FROM Author A
JOIN Book B ON A.aname = B.aname
ORDER BY A.aname;

-----------------------------------------------
-- Q1.3 BOOKS BY J.K. ROWLING
-----------------------------------------------
```

```
SELECT *
FROM Book
WHERE aname = 'J.K. Rowling';

-----------------------------------------------
-- Q1.4 UPDATE PHONE NUMBER
-----------------------------------------------

UPDATE Author
SET phone = '987654321'
WHERE aname = 'George Martin';

-----------------------------------------------
-- TRIGGER: CHECK AUTHOR EXISTS BEFORE INSERT BOOK
-----------------------------------------------

CREATE OR REPLACE FUNCTION check_author_exists()
RETURNS TRIGGER AS $$
BEGIN
   IF NOT EXISTS (
      SELECT 1 FROM Author WHERE aname = NEW.aname
   ) THEN
      RAISE EXCEPTION 'Invalid Name';
   END IF;

   RETURN NEW;
END;
```

```
$$ LANGUAGE plpgsql;

-- Remove old trigger if exists
DROP TRIGGER IF EXISTS before_book_insert ON Book;

CREATE TRIGGER before_book_insert
BEFORE INSERT ON Book
FOR EACH ROW
EXECUTE FUNCTION check_author_exists();

-----------------------------------------------
-- CURSOR FUNCTION: BOOKS OF SPECIFIC AUTHOR
-----------------------------------------------

CREATE OR REPLACE FUNCTION get_books_by_author(a_name
VARCHAR)
RETURNS VOID AS $$
DECLARE
   rec RECORD;

   book_cursor CURSOR FOR
      SELECT btitle, ISBN, category
      FROM Book
      WHERE aname = a_name;

BEGIN
   OPEN book_cursor;
```

```
    LOOP
        FETCH book_cursor INTO rec;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Title: %, ISBN: %, Category: %',
            rec.btitle, rec.ISBN, rec.category;
    END LOOP;

    CLOSE book_cursor;
END;
$$ LANGUAGE plpgsql;

-----------------------------------------------
-- CALL FUNCTION
-----------------------------------------------

SELECT get_books_by_author('J.K. Rowling');
```

Slip 16

Q1) Practical Questions on PostgresSQL [Total Marks 10]

Consider the following database

Branch (B_id, Brname, Brcity)

Customer (C_no, Cname, Caddress, City)

Loan_Application (L_no, L_amt_required, L_amt_approved,L_date)

Branch, Customer, Loan_Application are related with ternary relationship.

Ternary (B_id, C_no, L_no)

Assume appropriate data types for all the attributes.

Using above database, solve the following queries:

1. List the names of the customers for the "Aundh" branch.

2. Find the maximum loan amount approved.

3. Count the number of loan application received by "M.G. Road" branch.

4. Display the branch detail.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Create a view [10]

a. Display the name of customer

b. Display the average loan amount.

2. Write a stored function using cursors to accept a Loan date and find the customer with

branch name. [10]

Answer:

-- CREATE TABLES

```
CREATE TABLE Branch (
    B_id INT PRIMARY KEY,
    Brname VARCHAR(50),
    Brcity VARCHAR(50)
);

CREATE TABLE Customer (
    C_no INT PRIMARY KEY,
    Cname VARCHAR(50),
    Caddress VARCHAR(100),
    City VARCHAR(50)
);

CREATE TABLE Loan_Application (
    L_no INT PRIMARY KEY,
    L_amt_required NUMERIC(10,2),
    L_amt_approved NUMERIC(10,2),
    L_date DATE
);

-- Ternary relationship
CREATE TABLE Ternary (
    B_id INT,
    C_no INT,
    L_no INT,
    PRIMARY KEY (B_id, C_no, L_no),
    FOREIGN KEY (B_id) REFERENCES Branch(B_id),
```

```
    FOREIGN KEY (C_no) REFERENCES Customer(C_no),
    FOREIGN KEY (L_no) REFERENCES Loan_Application(L_no)
);

-----------------------------------------------
-- INSERT DATA
-----------------------------------------------

-- Branch
INSERT INTO Branch VALUES
(1, 'Aundh', 'Pune'),
(2, 'M.G. Road', 'Pune');

-- Customer
INSERT INTO Customer VALUES
(101, 'Amit', 'Street 1', 'Pune'),
(102, 'Neha', 'Street 2', 'Pune'),
(103, 'Rahul', 'Street 3', 'Mumbai'),
(104, 'Priya', 'Street 4', 'Pune');

-- Loan Application
INSERT INTO Loan_Application VALUES
(201, 50000, 45000, '2024-01-10'),
(202, 70000, 65000, '2024-02-15'),
(203, 90000, 85000, '2024-03-20'),
(204, 40000, 38000, '2024-04-05');
```

```
-- Ternary
INSERT INTO Ternary VALUES
(1, 101, 201),
(1, 102, 202),
(2, 103, 203),
(2, 104, 204);

------------------------------------------------
-- Q1.1 Customers for "Aundh" branch
------------------------------------------------

SELECT C.Cname
FROM Customer C
JOIN Ternary T ON C.C_no = T.C_no
JOIN Branch B ON B.B_id = T.B_id
WHERE B.Brname = 'Aundh';

------------------------------------------------
-- Q1.2 Maximum loan approved
------------------------------------------------

SELECT MAX(L_amt_approved) AS max_approved_amount
FROM Loan_Application;

------------------------------------------------
-- Q1.3 Count loan applications for M.G. Road
------------------------------------------------
```

```
SELECT COUNT(*) AS total_applications
FROM Ternary T
JOIN Branch B ON B.B_id = T.B_id
WHERE B.Brname = 'M.G. Road';

-----------------------------------------------
-- Q1.4 Display branch details
-----------------------------------------------

SELECT * FROM Branch;

-----------------------------------------------
-- Q2.1 VIEW: Customer name + average loan
-----------------------------------------------

CREATE OR REPLACE VIEW Customer_Loan_View AS
SELECT
   C.Cname,
   AVG(L.L_amt_approved) AS avg_loan_amount
FROM Customer C
JOIN Ternary T ON C.C_no = T.C_no
JOIN Loan_Application L ON L.L_no = T.L_no
GROUP BY C.Cname;
```

```
-----------------------------------------------
-- Q2.2 FUNCTION (CURSOR): Customer + Branch by date
-----------------------------------------------

CREATE OR REPLACE FUNCTION
get_customer_branch_by_date(input_date DATE)
RETURNS TABLE (customer_name VARCHAR, branch_name VARCHAR)
LANGUAGE plpgsql
AS $$
DECLARE
    rec RECORD;

    cur CURSOR FOR
        SELECT C.Cname, B.Brname
        FROM Customer C
        JOIN Ternary T ON C.C_no = T.C_no
        JOIN Branch B ON B.B_id = T.B_id
        JOIN Loan_Application L ON L.L_no = T.L_no
        WHERE L.L_date = input_date;

BEGIN
    OPEN cur;

    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;

        customer_name := rec.Cname;
```

```
        branch_name := rec.Brname;

        RETURN NEXT;
    END LOOP;

    CLOSE cur;
END;
$$;

-------------------------------------------------
-- FUNCTION CALL
-------------------------------------------------

SELECT * FROM get_customer_branch_by_date('2024-01-10');
```

Slip 17

Q1) Practical Questions on PostgresSQL [Total Marks 10]

Consider the following database

Patient: (pid,pname, age)

Doctor:(did,dname, specialization)

Relationship: A doctor can have many patients; each patient is assigned to exactly one doctor.

Using above database, solve the following queries:

1. List the patients name of doctor ‘Dr. Sharma’.

2. List doctors all details.

3. Update specialization of ‘Dr. Gupta’ to ‘Cardiologist’.

4. Delete all patients of ‘Dr.Agarwal’.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Create a View:

a. To display Doctor details who check patient ”Mr.P.V.Joshi”. [10]

b. To display Patient name with age.

2. Write a cursor to display doctor details along with patient. [10]

Answer:

```
-----------------------------------------------
-- CREATE TABLES
-----------------------------------------------

CREATE TABLE Doctor (
    did INT PRIMARY KEY,
    dname VARCHAR(50),
```

```
    specialization VARCHAR(50)
);

CREATE TABLE Patient (
    pid INT PRIMARY KEY,
    pname VARCHAR(50),
    age INT,
    did INT,
    FOREIGN KEY (did) REFERENCES Doctor(did) ON DELETE CASCADE
);

-----------------------------------------------
-- INSERT DATA INTO DOCTOR
-----------------------------------------------

INSERT INTO Doctor VALUES
(1, 'Dr. Sharma', 'General Physician'),
(2, 'Dr. Gupta', 'Neurologist'),
(3, 'Dr. Agarwal', 'Orthopedic'),
(4, 'Dr. Mehta', 'Pediatrician');

-----------------------------------------------
-- INSERT DATA INTO PATIENT
-----------------------------------------------

INSERT INTO Patient VALUES
(101, 'Mr. P.V. Joshi', 45, 1),
```

```
(102, 'Anita', 30, 1),
(103, 'Rohit', 25, 2),
(104, 'Suman', 50, 3);

-----------------------------------------------
-- Q1.1 Patients of Dr. Sharma
-----------------------------------------------

SELECT P.pname
FROM Patient P
JOIN Doctor D ON P.did = D.did
WHERE D.dname = 'Dr. Sharma';

-----------------------------------------------
-- Q1.2 All doctor details
-----------------------------------------------

SELECT * FROM Doctor;

-----------------------------------------------
-- Q1.3 Update specialization of Dr. Gupta
-----------------------------------------------

UPDATE Doctor
SET specialization = 'Cardiologist'
WHERE dname = 'Dr. Gupta';
```

```
-----------------------------------------------
-- Q1.4 Delete all patients of Dr. Agarwal
-----------------------------------------------

DELETE FROM Patient
WHERE did = (
    SELECT did FROM Doctor WHERE dname = 'Dr. Agarwal'
);

-----------------------------------------------
-- Q2.1 VIEW: Doctor who treated Mr. P.V. Joshi
-----------------------------------------------

CREATE OR REPLACE VIEW Doctor_For_Joshi AS
SELECT D.*
FROM Doctor D
JOIN Patient P ON D.did = P.did
WHERE P.pname = 'Mr. P.V. Joshi';

-----------------------------------------------
-- Q2.2 VIEW: Patient name with age
-----------------------------------------------

CREATE OR REPLACE VIEW Patient_Age_View AS
SELECT pname, age
FROM Patient;
```

```
-----------------------------------------------
-- CURSOR: Doctor + Patient details
-----------------------------------------------

DO $$
DECLARE
   rec RECORD;

   cur CURSOR FOR
      SELECT D.did, D.dname, D.specialization, P.pname, P.age
      FROM Doctor D
      JOIN Patient P ON D.did = P.did;

BEGIN
   OPEN cur;

   LOOP
      FETCH cur INTO rec;
      EXIT WHEN NOT FOUND;

      RAISE NOTICE 'Doctor ID: %, Name: %, Specialization: %, Patient: %,
Age: %',
         rec.did, rec.dname, rec.specialization, rec.pname, rec.age;
   END LOOP;

   CLOSE cur;
END $$;
```

```
-----------------------------------------------
-- OPTIONAL FUNCTION VERSION (CURSOR)
-----------------------------------------------

CREATE OR REPLACE FUNCTION show_doctor_patient()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
   rec RECORD;

   cur CURSOR FOR
      SELECT D.dname, D.specialization, P.pname
      FROM Doctor D
      JOIN Patient P ON D.did = P.did;

BEGIN
   OPEN cur;

   LOOP
      FETCH cur INTO rec;
      EXIT WHEN NOT FOUND;

      RAISE NOTICE 'Doctor: %, Specialization: %, Patient: %',
         rec.dname, rec.specialization, rec.pname;
   END LOOP;
```

```
    CLOSE cur;
END;
$$;


-----------------------------------------------
-- FUNCTION CALL
-----------------------------------------------

SELECT show_doctor_patient();
```

Slip 18

Q1) Practical Questions on PostgresSQL [Total Marks 10]

Consider the following database

Product: (pname, pid, price)

Order: (oid, orderdate, customer)

Relationship: A product can be in many orders; an order can contain many products.

Using above database, solve the following queries:

1. List products of order id 101.

2. List Product details.

3. Display product name and price.

4. Insert 2 records into product table.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Create a View:

a) To display product details. [10]

b) To display the order with the highest demand.

2. Write a function to display all details of product with customer . [10]

Answer:

```
-----------------------------------------------
-- CREATE TABLES
-----------------------------------------------

CREATE TABLE Product (
    pid INT PRIMARY KEY,
    pname VARCHAR(50),
    price NUMERIC(10,2)
);
```

```
CREATE TABLE Orders (
    oid INT PRIMARY KEY,
    orderdate DATE,
    customer VARCHAR(50)
);

-- Junction table (Many-to-Many)
CREATE TABLE Order_Product (
    oid INT,
    pid INT,
    PRIMARY KEY (oid, pid),
    FOREIGN KEY (oid) REFERENCES Orders(oid) ON DELETE CASCADE,
    FOREIGN KEY (pid) REFERENCES Product(pid) ON DELETE
CASCADE
);

-----------------------------------------------
-- INSERT DATA INTO PRODUCT
-----------------------------------------------

INSERT INTO Product VALUES
(1, 'Laptop', 60000),
(2, 'Mobile', 20000),
(3, 'Tablet', 30000),
(4, 'Headphones', 2000);

-----------------------------------------------
```

```
-- INSERT DATA INTO ORDERS
-----------------------------------------------

INSERT INTO Orders VALUES
(101, '2024-01-10', 'Amit'),
(102, '2024-02-15', 'Neha'),
(103, '2024-03-20', 'Rahul'),
(104, '2024-04-05', 'Priya');

-----------------------------------------------
-- RELATIONSHIP DATA
-----------------------------------------------

INSERT INTO Order_Product VALUES
(101, 1),
(101, 2),
(102, 3),
(103, 1),
(104, 4);

-----------------------------------------------
-- Q1.1 PRODUCTS OF ORDER 101
-----------------------------------------------

SELECT P.pname
FROM Product P
JOIN Order_Product OP ON P.pid = OP.pid
```

```
WHERE OP.oid = 101;

-----------------------------------------------
-- Q1.2 PRODUCT DETAILS
-----------------------------------------------

SELECT * FROM Product;

-----------------------------------------------
-- Q1.3 PRODUCT NAME AND PRICE
-----------------------------------------------

SELECT pname, price FROM Product;

-----------------------------------------------
-- Q1.4 INSERT 2 PRODUCTS
-----------------------------------------------

INSERT INTO Product VALUES
(5, 'Smartwatch', 8000),
(6, 'Camera', 45000);

-----------------------------------------------
-- Q2.1 VIEW: PRODUCT DETAILS
-----------------------------------------------

CREATE OR REPLACE VIEW Product_Details AS
```

```
SELECT * FROM Product;


-----------------------------------------------
-- Q2.2 VIEW: HIGHEST DEMAND ORDER
-----------------------------------------------

CREATE OR REPLACE VIEW Highest_Demand_Order AS
SELECT oid, COUNT(pid) AS total_products
FROM Order_Product
GROUP BY oid
HAVING COUNT(pid) = (
   SELECT MAX(cnt)
   FROM (
      SELECT COUNT(pid) AS cnt
      FROM Order_Product
      GROUP BY oid
   ) AS sub
);


-----------------------------------------------
-- Q2.3 FUNCTION: PRODUCT + CUSTOMER DETAILS
-----------------------------------------------

CREATE OR REPLACE FUNCTION get_product_customer_details()
RETURNS TABLE (
   product_name VARCHAR,
   price NUMERIC,
```

```
    customer VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT P.pname, P.price, O.customer
    FROM Product P
    JOIN Order_Product OP ON P.pid = OP.pid
    JOIN Orders O ON OP.oid = O.oid;
END;
$$;


-----------------------------------------------
-- FUNCTION CALL
-----------------------------------------------

SELECT * FROM get_product_customer_details();
```

Slip 19

Q1) Practical Questions on PostgresSQL [Total Marks 10]

Consider the following database

Student (sreg_no, sname , class)

Competition (c_no , cname , C_type)

The relationship is as follows:

STUDENT-COMPETITION: M-M with described attributes rank and year.

Class should be 'FYBCA' , 'SYBCA' and 'TYBCA'

Assume appropriate data types for all the attributes.

Using above database, solve the following queries:

1. List the names of all students who participated in Quiz Competition.

2. List the names of competitions.

3. Display the details of student whose name start with letter 'A'.

4. Insert 2 records into student table.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger after insert on student table to display message "All students are

welcome for competition" if student participate more than 2 competition. [10]

2. Write a stored function using cursors to accept registration number and find the

student participate in which competition. [10]

Answer:

```
-----------------------------------------------
-- CREATE TABLES
-----------------------------------------------

CREATE TABLE Student (
    sreg_no INT PRIMARY KEY,
    sname VARCHAR(50),
    class VARCHAR(10) CHECK (class IN ('FYBCA','SYBCA','TYBCA'))
);

CREATE TABLE Competition (
    c_no INT PRIMARY KEY,
    cname VARCHAR(50),
    c_type VARCHAR(50)
);

-- Many-to-Many relationship table
CREATE TABLE Student_Competition (
    sreg_no INT,
    c_no INT,
    rank INT,
    year INT,
    PRIMARY KEY (sreg_no, c_no),
```

```
    FOREIGN KEY (sreg_no) REFERENCES Student(sreg_no) ON DELETE
CASCADE,
    FOREIGN KEY (c_no) REFERENCES Competition(c_no) ON DELETE
CASCADE
);

-----------------------------------------------
-- INSERT DATA INTO STUDENT
-----------------------------------------------

INSERT INTO Student VALUES
(1, 'Amit', 'FYBCA'),
(2, 'Neha', 'SYBCA'),
(3, 'Anjali', 'TYBCA'),
(4, 'Rohit', 'FYBCA');

-----------------------------------------------
-- INSERT DATA INTO COMPETITION
-----------------------------------------------

INSERT INTO Competition VALUES
(101, 'Quiz Competition', 'Academic'),
(102, 'Coding Contest', 'Technical'),
(103, 'Debate', 'Cultural'),
(104, 'Hackathon', 'Technical');

-----------------------------------------------
-- INSERT PARTICIPATION DATA
```

```
-----------------------------------------------

INSERT INTO Student_Competition VALUES
(1, 101, 1, 2024),
(1, 102, 2, 2024),
(1, 103, 3, 2024),
(2, 101, 2, 2024),
(3, 104, 1, 2024),
(4, 103, 1, 2024);

-----------------------------------------------
-- Q1.1 Students in Quiz Competition
-----------------------------------------------

SELECT S.sname
FROM Student S
JOIN Student_Competition SC ON S.sreg_no = SC.sreg_no
JOIN Competition C ON SC.c_no = C.c_no
WHERE C.cname = 'Quiz Competition';

-----------------------------------------------
-- Q1.2 List competitions
-----------------------------------------------

SELECT cname FROM Competition;

-----------------------------------------------
```

```
-- Q1.3 Students starting with 'A'
-----------------------------------------------

SELECT *
FROM Student
WHERE sname LIKE 'A%';

-----------------------------------------------
-- Q1.4 Insert 2 students
-----------------------------------------------

INSERT INTO Student VALUES
(5, 'Ajay', 'SYBCA'),
(6, 'Sneha', 'TYBCA');

-----------------------------------------------
-- TRIGGER FUNCTION
-- After insert in Student_Competition
-----------------------------------------------

CREATE OR REPLACE FUNCTION check_participation()
RETURNS TRIGGER AS $$
DECLARE
   comp_count INT;
BEGIN
   SELECT COUNT(*) INTO comp_count
   FROM Student_Competition
```

```
    WHERE sreg_no = NEW.sreg_no;

    IF comp_count > 2 THEN
        RAISE NOTICE 'All students are welcome for competition';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-----------------------------------------------
-- CREATE TRIGGER
-----------------------------------------------

DROP TRIGGER IF EXISTS trg_check_participation ON
Student_Competition;

CREATE TRIGGER trg_check_participation
AFTER INSERT ON Student_Competition
FOR EACH ROW
EXECUTE FUNCTION check_participation();

-----------------------------------------------
-- STORED FUNCTION USING CURSOR
-----------------------------------------------

CREATE OR REPLACE FUNCTION get_student_competitions(input_reg
INT)
```

```
RETURNS TABLE (
    student_name VARCHAR,
    competition_name VARCHAR
)
LANGUAGE plpgsql
AS $$
DECLARE
    rec RECORD;

    cur CURSOR FOR
        SELECT S.sname, C.cname
        FROM Student S
        JOIN Student_Competition SC ON S.sreg_no = SC.sreg_no
        JOIN Competition C ON SC.c_no = C.c_no
        WHERE S.sreg_no = input_reg;

BEGIN
    OPEN cur;

    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;

        student_name := rec.sname;
        competition_name := rec.cname;

        RETURN NEXT;
```

```
    END LOOP;

    CLOSE cur;
END;
$$;

-----------------------------------------------
-- FUNCTION CALL
-----------------------------------------------

SELECT * FROM get_student_competitions(1);
```

Slip 20

Q.1) Practical Questions on PostgresSQL [Total Marks 10]

Consider the following database

Property (pno, description, area)

Owner (oname, address, phone)

An owner can have one or more properties, but a property belongs to exactly one owner. Create

the relations accordingly, so that the relationship is handled properly and the relations are

handled properly and the relations are in normalized form (3NF).

Using above database, solve the following queries:

1. List details of property where area is 'Moshi'

2. List the owners detail.

3. List property details owned by 'Mr. Kadam'

4. Update phone no of 'Mr. Patil' to 987842621

Q2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger before inserting description into a Property table; that area should in

Pune. Display appropriate message. [10]

2. Write a cursor to list the details of all Owners. [10]

Answer:

-- CREATE TABLES (3NF STRUCTURE)

```
CREATE TABLE Owner (
    oname VARCHAR(50) PRIMARY KEY,
    address TEXT,
    phone VARCHAR(15)
);

CREATE TABLE Property (
    pno INT PRIMARY KEY,
    description TEXT,
    area VARCHAR(50),
    oname VARCHAR(50),
    FOREIGN KEY (oname) REFERENCES Owner(oname)
);

-----------------------------------------------
-- INSERT DATA INTO OWNER
-----------------------------------------------

INSERT INTO Owner VALUES
('Mr. Kadam', 'Pune', '9876543210'),
('Mr. Patil', 'Mumbai', '9123456780'),
('Mr. Sharma', 'Delhi', '9988776655'),
('Mr. Joshi', 'Pune', '9090909090');

-----------------------------------------------
-- INSERT DATA INTO PROPERTY
-----------------------------------------------
```

```
INSERT INTO Property VALUES
(1, '2BHK Flat', 'Moshi', 'Mr. Kadam'),
(2, '3BHK Flat', 'Pimpri', 'Mr. Kadam'),
(3, 'Villa', 'Moshi', 'Mr. Patil'),
(4, '1BHK Flat', 'Nigdi', 'Mr. Patil'),
(5, 'Shop', 'Moshi', 'Mr. Sharma'),
(6, 'Office', 'Shivajinagar', 'Mr. Joshi'),
(7, 'Plot', 'Moshi', 'Mr. Kadam'),
(8, 'Bungalow', 'Aundh', 'Mr. Joshi');

-----------------------------------------------
-- Q1.1 PROPERTY IN MOSHI
-----------------------------------------------

SELECT *
FROM Property
WHERE area = 'Moshi';

-----------------------------------------------
-- Q1.2 OWNER DETAILS
-----------------------------------------------

SELECT * FROM Owner;

-----------------------------------------------
-- Q1.3 PROPERTIES OF MR. KADAM
```

```
-----------------------------------------------


SELECT *
FROM Property
WHERE oname = 'Mr. Kadam';


-----------------------------------------------
-- Q1.4 UPDATE PHONE NUMBER
-----------------------------------------------


UPDATE Owner
SET phone = '987842621'
WHERE oname = 'Mr. Patil';


-----------------------------------------------
-- TRIGGER FUNCTION
-- Allow only Pune area properties
-----------------------------------------------


CREATE OR REPLACE FUNCTION check_area()
RETURNS TRIGGER AS $$
BEGIN
   IF NEW.area NOT IN ('Moshi', 'Pimpri', 'Nigdi', 'Shivajinagar', 'Aundh')
THEN
      RAISE EXCEPTION 'Area must be in Pune region only!';
   END IF;


   RETURN NEW;
```

```
END;
$$ LANGUAGE plpgsql;

-----------------------------------------------
-- CREATE TRIGGER
-----------------------------------------------

DROP TRIGGER IF EXISTS trg_check_area ON Property;

CREATE TRIGGER trg_check_area
BEFORE INSERT ON Property
FOR EACH ROW
EXECUTE FUNCTION check_area();

-----------------------------------------------
-- CURSOR: DISPLAY ALL OWNERS
-----------------------------------------------

DO $$
DECLARE
   rec RECORD;

   owner_cursor CURSOR FOR
      SELECT * FROM Owner;

BEGIN
   OPEN owner_cursor;
```

```
    LOOP
        FETCH owner_cursor INTO rec;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Name: %, Address: %, Phone: %',
            rec.oname, rec.address, rec.phone;
    END LOOP;

    CLOSE owner_cursor;
END $$;
```

Slip 21

Q.1) Practical Questions on PostgresSQL [Total Marks 10]

Consider the following database of Movie, Actor and Producers,

Movie (m_name varchar (25), release_year integer, budget money)

Actor (a_name char (30), city varchar(30))

Producer (producer_id integer, pname char (30), p_address varchar (30))

Relationship: Movie–Actor Relationship (Many-to-Many)

Movie–Producer Relationship (Many-to-One)

1. List all movies released in the year 2015.

2. Find the names of all actors who live in 'Mumbai'.

3. Display the name and budget of movies .

4. Insert 2 records into actor table.

.

Q.2) Using above database solve following questions: [Total Marks 20]

1. Write a trigger before inserting record into movie table, check release_year should not be greater than current year. Display appropriate message. [10]

2. Write a cursor using function to list movie-wise charges of ‘Amitabh Bachchan’. [10]

Answer:

```
-----------------------------------------------

-- CREATE TABLES

-----------------------------------------------


CREATE TABLE Producer (
```

```
    producer_id INT PRIMARY KEY,
    pname VARCHAR(30),
    p_address VARCHAR(30)
);

CREATE TABLE Movie (
    m_name VARCHAR(25) PRIMARY KEY,
    release_year INT,
    budget MONEY,
    producer_id INT,
    FOREIGN KEY (producer_id) REFERENCES Producer(producer_id)
);

CREATE TABLE Actor (
    a_name VARCHAR(30) PRIMARY KEY,
    city VARCHAR(30)
);

-- Many-to-Many relationship
CREATE TABLE Movie_Actor (
    m_name VARCHAR(25),
    a_name VARCHAR(30),
    charges MONEY,
    PRIMARY KEY (m_name, a_name),
    FOREIGN KEY (m_name) REFERENCES Movie(m_name) ON DELETE
CASCADE,
    FOREIGN KEY (a_name) REFERENCES Actor(a_name) ON DELETE
CASCADE
```

```
);

-----------------------------------------------
-- INSERT DATA INTO PRODUCER
-----------------------------------------------

INSERT INTO Producer VALUES
(1, 'Karan Johar', 'Mumbai'),
(2, 'Aditya Chopra', 'Mumbai'),
(3, 'Rakesh Roshan', 'Mumbai');

-----------------------------------------------
-- INSERT DATA INTO MOVIE
-----------------------------------------------

INSERT INTO Movie VALUES
('Movie1', 2015, 5000000, 1),
('Movie2', 2016, 7000000, 2),
('Movie3', 2015, 8000000, 1),
('Movie4', 2018, 6000000, 3);

-----------------------------------------------
-- INSERT DATA INTO ACTOR
-----------------------------------------------

INSERT INTO Actor VALUES
('Amitabh Bachchan', 'Mumbai'),
```

```
('Shah Rukh Khan', 'Mumbai'),
('Salman Khan', 'Mumbai'),
('Aamir Khan', 'Delhi');

-----------------------------------------------
-- INSERT DATA INTO MOVIE_ACTOR
-----------------------------------------------

INSERT INTO Movie_Actor VALUES
('Movie1', 'Amitabh Bachchan', 2000000),
('Movie1', 'Shah Rukh Khan', 2500000),
('Movie2', 'Salman Khan', 3000000),
('Movie3', 'Amitabh Bachchan', 2200000),
('Movie4', 'Aamir Khan', 2700000),
('Movie2', 'Amitabh Bachchan', 2100000),
('Movie3', 'Salman Khan', 2300000),
('Movie4', 'Shah Rukh Khan', 2600000);

-----------------------------------------------
-- Q1.1 MOVIES RELEASED IN 2015
-----------------------------------------------

SELECT *
FROM Movie
WHERE release_year = 2015;

-----------------------------------------------
```

```
-- Q1.2 ACTORS FROM MUMBAI
-----------------------------------------------

SELECT a_name
FROM Actor
WHERE city = 'Mumbai';

-----------------------------------------------
-- Q1.3 MOVIE NAME AND BUDGET
-----------------------------------------------

SELECT m_name, budget
FROM Movie;

-----------------------------------------------
-- Q1.4 INSERT 2 ACTORS
-----------------------------------------------

INSERT INTO Actor VALUES
('Ranveer Singh', 'Mumbai'),
('Hrithik Roshan', 'Mumbai');

-----------------------------------------------
-- TRIGGER: VALIDATE RELEASE YEAR
-----------------------------------------------

CREATE OR REPLACE FUNCTION check_release_year()
```

```
RETURNS TRIGGER AS $$
BEGIN
   IF NEW.release_year > EXTRACT(YEAR FROM CURRENT_DATE)
THEN
      RAISE EXCEPTION 'Release year cannot be greater than current year!';
   END IF;

   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_release_year ON Movie;

CREATE TRIGGER trg_release_year
BEFORE INSERT ON Movie
FOR EACH ROW
EXECUTE FUNCTION check_release_year();

-----------------------------------------------
-- CURSOR FUNCTION: AMITABH CHARGES
-----------------------------------------------

CREATE OR REPLACE FUNCTION get_charges_amitabh()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
   rec RECORD;
```

```
    cur CURSOR FOR
        SELECT m_name, charges
        FROM Movie_Actor
        WHERE a_name = 'Amitabh Bachchan';

BEGIN
    OPEN cur;

    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Movie: %, Charges: %',
            rec.m_name, rec.charges;
    END LOOP;

    CLOSE cur;
END;
$$;

-----------------------------------------------
-- CALL FUNCTION
-----------------------------------------------

SELECT get_charges_amitabh();
```

Slip 22

Q.1) Practical Questions on PostgresSQL [Total Marks :10]

Consider the following database of Student, Course and Enrollment,

Student(student_id, name ,age)

Course (course_id , course_name, credits)

Enrollment (student_id, course_id, enrollment_date)

Relationship: Student–Enrollment (One-to-Many)

Course–Enrollment (One-to-Many)

Many-to-Many via Enrollment table

1. List the names of all students enrolled in B.Sc(Comp.Sci) course.

2. Show all courses with more than 3 credits.

3. Insert 2 reocords into course table.

4. Display student enrollment date of 'Ram'.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a function to display count of student

[10]

2. Create a view that shows each student's name and the courses they are enrolled in. [10]

Answer:

```
-----------------------------------------------
-- CREATE TABLES
-----------------------------------------------

CREATE TABLE Student (
    student_id INT PRIMARY KEY,
```

```
    name VARCHAR(50),
    age INT
);

CREATE TABLE Course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50),
    credits INT
);

CREATE TABLE Enrollment (
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES Student(student_id) ON
DELETE CASCADE,
    FOREIGN KEY (course_id) REFERENCES Course(course_id) ON
DELETE CASCADE
);

-----------------------------------------------
-- INSERT DATA INTO STUDENT
-----------------------------------------------

INSERT INTO Student VALUES
(1, 'Ram', 20),
(2, 'Shyam', 21),
```

```
(3, 'Amit', 22),
(4, 'Neha', 20);

-----------------------------------------------
-- INSERT DATA INTO COURSE
-----------------------------------------------

INSERT INTO Course VALUES
(101, 'B.Sc(Comp.Sci)', 4),
(102, 'BCA', 3),
(103, 'BBA', 3),
(104, 'MCA', 5);

-----------------------------------------------
-- INSERT DATA INTO ENROLLMENT
-----------------------------------------------

INSERT INTO Enrollment VALUES
(1, 101, '2024-06-01'),
(2, 101, '2024-06-02'),
(3, 102, '2024-06-03'),
(4, 103, '2024-06-04'),
(1, 104, '2024-06-05'),
(2, 103, '2024-06-06'),
(3, 101, '2024-06-07'),
(4, 104, '2024-06-08');
```

```
-----------------------------------------------
-- Q1.1 STUDENTS IN B.Sc(Comp.Sci)
-----------------------------------------------

SELECT s.name
FROM Student s
JOIN Enrollment e ON s.student_id = e.student_id
JOIN Course c ON e.course_id = c.course_id
WHERE c.course_name = 'B.Sc(Comp.Sci)';

-----------------------------------------------
-- Q1.2 COURSES WITH MORE THAN 3 CREDITS
-----------------------------------------------

SELECT *
FROM Course
WHERE credits > 3;

-----------------------------------------------
-- Q1.3 INSERT 2 COURSES
-----------------------------------------------

INSERT INTO Course VALUES
(105, 'MBA', 4),
(106, 'B.Com', 3);

-----------------------------------------------
```

```
-- Q1.4 ENROLLMENT DATE OF RAM
-----------------------------------------------

SELECT e.enrollment_date
FROM Enrollment e
JOIN Student s ON e.student_id = s.student_id
WHERE s.name = 'Ram';

-----------------------------------------------
-- Q2.1 FUNCTION: COUNT OF STUDENTS
-----------------------------------------------

CREATE OR REPLACE FUNCTION count_students()
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
   total INT;
BEGIN
   SELECT COUNT(*) INTO total FROM Student;
   RETURN total;
END;
$$;

-----------------------------------------------
-- FUNCTION CALL
-----------------------------------------------
```

```
SELECT count_students();

-----------------------------------------------
-- Q2.2 VIEW: STUDENT + COURSE DETAILS
-----------------------------------------------

CREATE OR REPLACE VIEW student_course_view AS
SELECT s.name, c.course_name
FROM Student s
JOIN Enrollment e ON s.student_id = e.student_id
JOIN Course c ON e.course_id = c.course_id;

-----------------------------------------------
-- VIEW OUTPUT
-----------------------------------------------

SELECT * FROM student_course_view;
```

Slip 23

Q.1) Practical Questions on PostgresSQL [Total Marks: 10]

Consider the following database of Bus, Route and Driver,

Bus (bus_no int ,b_capacity int , depot_name varchar(20))

Route (route_no int, source char (20), destination char (20), no_of_stations int)

Driver (driver_no int ,driver_name char(20), license_no int, address char(20), d_age int , salary float)

Relationship: Bus – Driver (One-to-One or One-to-Many)

Bus – Route (Many-to-One)

1. Display all buses with a capacity greater than 50

2. List all routes starting from 'Delhi'

3. Display the names and license numbers of all drivers .

4. Find all drivers older than 50 years

Q.2) Using above database solve following questions: [Total Marks :20]

1. Write a stored function to accept depot name and display driver details having age more than 50. [10]

2. Write a trigger which will prevent deleting drivers living in ‘Pune’ [10]

Answer:

-- CREATE TABLES

```
CREATE TABLE Bus (
    bus_no INT PRIMARY KEY,
    b_capacity INT,
    depot_name VARCHAR(20)
);

CREATE TABLE Driver (
    driver_no INT PRIMARY KEY,
    driver_name VARCHAR(20),
    license_no INT,
    address VARCHAR(20),
    d_age INT,
    salary FLOAT,
    bus_no INT,
    FOREIGN KEY (bus_no) REFERENCES Bus(bus_no) ON DELETE
CASCADE
);

CREATE TABLE Route (
    route_no INT PRIMARY KEY,
    source VARCHAR(20),
    destination VARCHAR(20),
    no_of_stations INT,
    bus_no INT,
    FOREIGN KEY (bus_no) REFERENCES Bus(bus_no) ON DELETE
CASCADE
);
```

```
-----------------------------------------------
-- INSERT DATA INTO BUS
-----------------------------------------------

INSERT INTO Bus VALUES
(1, 60, 'Pune Depot'),
(2, 45, 'Mumbai Depot'),
(3, 55, 'Delhi Depot'),
(4, 70, 'Pune Depot');

-----------------------------------------------
-- INSERT DATA INTO ROUTE
-----------------------------------------------

INSERT INTO Route VALUES
(101, 'Delhi', 'Agra', 10, 1),
(102, 'Mumbai', 'Pune', 8, 2),
(103, 'Delhi', 'Jaipur', 12, 3),
(104, 'Pune', 'Nashik', 9, 4);

-----------------------------------------------
-- INSERT DATA INTO DRIVER
-----------------------------------------------

INSERT INTO Driver VALUES
(1, 'Ramesh', 1111, 'Pune', 55, 30000, 1),
(2, 'Suresh', 2222, 'Mumbai', 45, 28000, 2),
```

```
(3, 'Mahesh', 3333, 'Pune', 60, 35000, 3),
(4, 'Rajesh', 4444, 'Delhi', 52, 32000, 4),
(5, 'Amit', 5555, 'Pune', 48, 27000, 1),
(6, 'Vijay', 6666, 'Nagpur', 58, 31000, 2),
(7, 'Kiran', 7777, 'Pune', 51, 29000, 3),
(8, 'Sunil', 8888, 'Delhi', 42, 26000, 4);

-----------------------------------------------
-- Q1.1 BUSES WITH CAPACITY > 50
-----------------------------------------------

SELECT *
FROM Bus
WHERE b_capacity > 50;

-----------------------------------------------
-- Q1.2 ROUTES FROM DELHI
-----------------------------------------------

SELECT *
FROM Route
WHERE source = 'Delhi';

-----------------------------------------------
-- Q1.3 DRIVER DETAILS
-----------------------------------------------
```

```
SELECT driver_name, license_no
FROM Driver;


-------------------------------------------------
-- Q1.4 DRIVERS OLDER THAN 50
-------------------------------------------------

SELECT *
FROM Driver
WHERE d_age > 50;


-------------------------------------------------
-- Q2.1 FUNCTION: DRIVERS >50 BY DEPOT
-------------------------------------------------

CREATE OR REPLACE FUNCTION get_drivers_by_depot(dep_name
VARCHAR)
RETURNS TABLE (
    driver_name VARCHAR,
    d_age INT,
    address VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT d.driver_name, d.d_age, d.address
    FROM Driver d
```

```
    JOIN Bus b ON d.bus_no = b.bus_no
    WHERE b.depot_name = dep_name
      AND d.d_age > 50;
END;
$$;

-----------------------------------------------
-- FUNCTION CALL
-----------------------------------------------

SELECT * FROM get_drivers_by_depot('Pune Depot');

-----------------------------------------------
-- Q2.2 TRIGGER: PREVENT DELETE DRIVERS FROM PUNE
-----------------------------------------------

CREATE OR REPLACE FUNCTION prevent_delete_pune()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.address = 'Pune' THEN
        RAISE EXCEPTION 'Cannot delete drivers from Pune!';
    END IF;

    RETURN NEW;
END;
```

```
$$;

-- FIX: correct OLD usage + trigger binding

DROP TRIGGER IF EXISTS trg_prevent_delete ON Driver;

CREATE TRIGGER trg_prevent_delete
BEFORE DELETE ON Driver
FOR EACH ROW
EXECUTE FUNCTION prevent_delete_pune();
```

Slip 24

Q.1) Practical Questions on PostgresSQL [Total Marks :10]

Consider the following database of Branch, Customer and Loan_Application,

Branch (bid integer, brname char (30), brcity char (10))

Customer (cno integer, cname char (20), caddr char (35), city (20))

Loan_Application (lno integer, lamtrequired money, lamtapproved money, l_date date)

Relationship: Customer & Branch linked via Loan_Application (many-to-one each)

1. List the names of the customers who have received loan less than their requirement.

2. Find the maximum loan amount approved.

3. Find out the total loan amount sanctioned by "Deccan "branch.

4. Count the number of loan applications received by "M.G.ROAD" branch.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Create a view to display the details of all customers who have received loan amount less than

their requirement. [10]

2. Write a stored function to display customer details who have applied for loan more than one

branches. [10]

Answer:

```
-----------------------------------------------

-- CREATE TABLES

-----------------------------------------------


CREATE TABLE Branch (

    bid INTEGER PRIMARY KEY,
```

```
    brname VARCHAR(30),
    brcity VARCHAR(20)
);

CREATE TABLE Customer (
    cno INTEGER PRIMARY KEY,
    cname VARCHAR(20),
    caddr VARCHAR(35),
    city VARCHAR(20)
);

CREATE TABLE Loan_Application (
    lno INTEGER PRIMARY KEY,
    cno INTEGER,
    bid INTEGER,
    lamtrequired MONEY,
    lamtapproved MONEY,
    l_date DATE,
    FOREIGN KEY (cno) REFERENCES Customer(cno) ON DELETE
CASCADE,
    FOREIGN KEY (bid) REFERENCES Branch(bid) ON DELETE CASCADE
);

-----------------------------------------------
-- INSERT DATA INTO BRANCH
-----------------------------------------------

INSERT INTO Branch VALUES
```

```
(1, 'Deccan', 'Pune'),
(2, 'M.G.ROAD', 'Pune'),
(3, 'ShivajiNagar', 'Pune');

-----------------------------------------------
-- INSERT DATA INTO CUSTOMER
-----------------------------------------------

INSERT INTO Customer VALUES
(101, 'Amit', 'Pune Camp', 'Pune'),
(102, 'Neha', 'Kothrud', 'Pune'),
(103, 'Rahul', 'Baner', 'Pune'),
(104, 'Sneha', 'Aundh', 'Pune');

-----------------------------------------------
-- INSERT DATA INTO LOAN_APPLICATION
-----------------------------------------------

INSERT INTO Loan_Application VALUES
(1, 101, 1, 50000, 40000, '2026-01-10'),
(2, 102, 2, 70000, 70000, '2026-02-12'),
(3, 103, 1, 60000, 50000, '2026-03-15'),
(4, 104, 3, 80000, 60000, '2026-03-20'),
(5, 101, 2, 30000, 20000, '2026-04-01'),
(6, 102, 1, 90000, 85000, '2026-04-05'),
(7, 103, 2, 40000, 30000, '2026-04-10'),
(8, 104, 1, 100000, 95000, '2026-04-15');
```

```
-----------------------------------------------
-- Q1.1 CUSTOMERS WHO GOT LESS LOAN
-----------------------------------------------

SELECT c.cname
FROM Customer c
JOIN Loan_Application l ON c.cno = l.cno
WHERE l.lamtapproved < l.lamtrequired;

-----------------------------------------------
-- Q1.2 MAX LOAN APPROVED
-----------------------------------------------

SELECT MAX(lamtapproved) AS max_loan
FROM Loan_Application;

-----------------------------------------------
-- Q1.3 TOTAL LOAN OF DECCAN BRANCH
-----------------------------------------------

SELECT SUM(l.lamtapproved) AS total_loan
FROM Loan_Application l
JOIN Branch b ON l.bid = b.bid
WHERE b.brname = 'Deccan';

-----------------------------------------------
```

```
-- Q1.4 COUNT LOANS AT M.G.ROAD
-----------------------------------------------

SELECT COUNT(*) AS total_applications
FROM Loan_Application l
JOIN Branch b ON l.bid = b.bid
WHERE b.brname = 'M.G.ROAD';

-----------------------------------------------
-- Q2.1 VIEW: LOANS LESS THAN REQUIRED
-----------------------------------------------

CREATE OR REPLACE VIEW Customer_Loan_View AS
SELECT
    c.cno,
    c.cname,
    c.city,
    l.lamtrequired,
    l.lamtapproved
FROM Customer c
JOIN Loan_Application l ON c.cno = l.cno
WHERE l.lamtapproved < l.lamtrequired;

-----------------------------------------------
-- VIEW OUTPUT
-----------------------------------------------
```

```
SELECT * FROM Customer_Loan_View;


-----------------------------------------------
-- Q2.2 FUNCTION: CUSTOMERS IN MULTIPLE BRANCHES
-----------------------------------------------

CREATE OR REPLACE FUNCTION multi_branch_customers()
RETURNS TABLE (
    cno INTEGER,
    cname VARCHAR,
    branch_count BIGINT
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        c.cno,
        c.cname,
        COUNT(DISTINCT l.bid)
    FROM Customer c
    JOIN Loan_Application l ON c.cno = l.cno
    GROUP BY c.cno, c.cname
    HAVING COUNT(DISTINCT l.bid) > 1;
END;
$$;
```

```
-----------------------------------------------
-- FUNCTION CALL
-----------------------------------------------

SELECT * FROM multi_branch_customers();
```

Slip 25

Q.1) Practical Questions on PostgresSQL [Total Marks: 10]

Consider the following database of Salesman, Trip, Dept and Expense,

Salesman (sno integer, s_name varchar (30), start_year integer)

Trip (tno integer, from_citychar (20), to_citychar (20), departure_date date, return_date date)

Dept (dept_no varchar (10), dept_name char (20))

Expense (eid integer, amount money)

Relationship: Trip → Salesman (many-to-one)

Expense → Trip (many-to-one)

Salesman → Dept (many-to-one)

1. List all details of salesman.

2. Show all trips that started from 'Mumbai'

3. Display the names of all departments.

4. List all expenses greater than 1,000.

Q.2) Using above database solve following questions: [Total Marks :20]

1. Write a cursor to display trip details which has maximum expenses. [10]

2. Write a stored function to count the total number of business trips from Pune to Mumbai. [10]

Answer:

```
-----------------------------------------------
-- CREATE TABLES
-----------------------------------------------

CREATE TABLE Dept (
    dept_no VARCHAR(10) PRIMARY KEY,
```

```
    dept_name VARCHAR(20)
);

CREATE TABLE Salesman (
    sno INTEGER PRIMARY KEY,
    s_name VARCHAR(30),
    start_year INTEGER,
    dept_no VARCHAR(10),
    FOREIGN KEY (dept_no) REFERENCES Dept(dept_no) ON DELETE
CASCADE
);

CREATE TABLE Trip (
    tno INTEGER PRIMARY KEY,
    sno INTEGER,
    from_city VARCHAR(20),
    to_city VARCHAR(20),
    departure_date DATE,
    return_date DATE,
    FOREIGN KEY (sno) REFERENCES Salesman(sno) ON DELETE
CASCADE
);

CREATE TABLE Expense (
    eid INTEGER PRIMARY KEY,
    tno INTEGER,
    amount MONEY,
    FOREIGN KEY (tno) REFERENCES Trip(tno) ON DELETE CASCADE
```

```
);

-----------------------------------------------
-- INSERT DATA INTO DEPT
-----------------------------------------------

INSERT INTO Dept VALUES
('D1', 'Sales'),
('D2', 'Marketing'),
('D3', 'HR');

-----------------------------------------------
-- INSERT DATA INTO SALESMAN
-----------------------------------------------

INSERT INTO Salesman VALUES
(101, 'Amit', 2018, 'D1'),
(102, 'Neha', 2019, 'D2'),
(103, 'Rahul', 2020, 'D1'),
(104, 'Sneha', 2021, 'D3');

-----------------------------------------------
-- INSERT DATA INTO TRIP
-----------------------------------------------

INSERT INTO Trip VALUES
(1, 101, 'Mumbai', 'Pune', '2026-01-10', '2026-01-12'),
```

```
(2, 102, 'Pune', 'Mumbai', '2026-02-05', '2026-02-07'),
(3, 103, 'Mumbai', 'Delhi', '2026-02-15', '2026-02-20'),
(4, 104, 'Pune', 'Mumbai', '2026-03-01', '2026-03-03'),
(5, 101, 'Mumbai', 'Goa', '2026-03-10', '2026-03-15'),
(6, 102, 'Delhi', 'Mumbai', '2026-03-20', '2026-03-25'),
(7, 103, 'Pune', 'Mumbai', '2026-04-01', '2026-04-04'),
(8, 104, 'Mumbai', 'Chennai', '2026-04-10', '2026-04-15');

------------------------------------------------
-- INSERT DATA INTO EXPENSE
------------------------------------------------

INSERT INTO Expense VALUES
(1, 1, 1200),
(2, 2, 900),
(3, 3, 1500),
(4, 4, 800),
(5, 5, 2000),
(6, 6, 1100),
(7, 7, 1300),
(8, 8, 700);

------------------------------------------------
-- Q1.1 ALL SALESMAN DETAILS
------------------------------------------------

SELECT * FROM Salesman;
```

```
-----------------------------------------------
-- Q1.2 TRIPS FROM MUMBAI
-----------------------------------------------

SELECT *
FROM Trip
WHERE from_city = 'Mumbai';

-----------------------------------------------
-- Q1.3 ALL DEPARTMENTS
-----------------------------------------------

SELECT dept_name
FROM Dept;

-----------------------------------------------
-- Q1.4 EXPENSES > 1000
-----------------------------------------------

SELECT *
FROM Expense
WHERE amount::numeric > 1000;

-----------------------------------------------
-- Q2.1 CURSOR: TRIP WITH MAX EXPENSE
-----------------------------------------------
```

```
DO $$
DECLARE
    rec RECORD;
    max_exp MONEY;

    cur CURSOR FOR
        SELECT t.*
        FROM Trip t
        JOIN Expense e ON t.tno = e.tno
        WHERE e.amount = max_exp;

BEGIN
    -- find maximum expense
    SELECT MAX(amount) INTO max_exp FROM Expense;

    OPEN cur;

    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Trip No: %, From: %, To: %, Departure: %, Return: %',
            rec.tno, rec.from_city, rec.to_city, rec.departure_date, rec.return_date;
    END LOOP;

    CLOSE cur;
```

```
END $$;

-----------------------------------------------
-- Q2.2 FUNCTION: PUNE → MUMBAI TRIPS COUNT
-----------------------------------------------

CREATE OR REPLACE FUNCTION count_pune_to_mumbai()
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
   trip_count INTEGER;
BEGIN
   SELECT COUNT(*) INTO trip_count
   FROM Trip
   WHERE from_city = 'Pune'
     AND to_city = 'Mumbai';

   RETURN trip_count;
END;
$$;

-----------------------------------------------
-- FUNCTION CALL
-----------------------------------------------

SELECT count_pune_to_mumbai();
```

Slip 26

Q.1) Practical Questions on PostgresSQL [Total Marks: 10]

Consider the following database of Train, Passenger and Ticket,

Train (Train_no int, train_name varchar (20), depart_time time , arrival_time time, source_stn

varchar (20),dest_stn varchar (20), no_of_res_bogies int ,bogie_capacity int)

Passenger (Passenger_id int, passenger_name varchar (20), address varchar (30), age int, gender char)

Ticket(train_no int, passenger_id int, ticket_no int ,bogie_no int, no_of_berths int ,

tdate date , ticket_amt decimal(7,2),status char)

Relationship: Train to Ticket (One to Many)

Passenger to Ticket (One to Many)

1. List all the train names along with their departure and arrival times.

2. Find the names and addresses of all passengers who are older than 60.

3. Display the source and destination stations of all trains that have more than 10 reserved bogies.

4. Display the names of all male passengers.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a trigger to restrict the bogie capacity of any train to 30. [10]

2. Write a stored function to calculate the ticket amount paid by all the passengers on 12/12/2019 for all the trains. [10]

Answer:

-- CREATE TABLES

```
CREATE TABLE Train (
    train_no INT PRIMARY KEY,
    train_name VARCHAR(20),
    depart_time TIME,
    arrival_time TIME,
    source_stn VARCHAR(20),
    dest_stn VARCHAR(20),
    no_of_res_bogies INT,
    bogie_capacity INT
);

CREATE TABLE Passenger (
    passenger_id INT PRIMARY KEY,
    passenger_name VARCHAR(20),
    address VARCHAR(30),
    age INT,
    gender CHAR(1)
);

CREATE TABLE Ticket (
    ticket_no INT PRIMARY KEY,
    train_no INT,
    passenger_id INT,
    bogie_no INT,
    no_of_berths INT,
    tdate DATE,
```

```
    ticket_amt DECIMAL(7,2),
    status CHAR(1),
    FOREIGN KEY (train_no) REFERENCES Train(train_no) ON DELETE CASCADE,
    FOREIGN KEY (passenger_id) REFERENCES Passenger(passenger_id) ON DELETE CASCADE
);

-----------------------------------------------
-- INSERT DATA INTO TRAIN
-----------------------------------------------

INSERT INTO Train VALUES
(101, 'Express1', '08:00', '12:00', 'Pune', 'Mumbai', 12, 30),
(102, 'Express2', '09:00', '14:00', 'Mumbai', 'Delhi', 8, 25),
(103, 'Express3', '07:30', '11:30', 'Pune', 'Nagpur', 15, 30),
(104, 'Express4', '10:00', '18:00', 'Delhi', 'Chennai', 20, 30);

-----------------------------------------------
-- INSERT DATA INTO PASSENGER
-----------------------------------------------

INSERT INTO Passenger VALUES
(1, 'Amit', 'Pune', 65, 'M'),
(2, 'Neha', 'Mumbai', 30, 'F'),
(3, 'Rahul', 'Delhi', 70, 'M'),
(4, 'Sneha', 'Chennai', 55, 'F');
```

```
-----------------------------------------------
-- INSERT DATA INTO TICKET
-----------------------------------------------

INSERT INTO Ticket VALUES
(1001, 101, 1, 1, 2, '2019-12-12', 1500.00, 'C'),
(1002, 102, 2, 2, 1, '2019-12-12', 1200.00, 'C'),
(1003, 103, 3, 3, 2, '2019-12-12', 1800.00, 'C'),
(1004, 104, 4, 4, 1, '2019-12-12', 2000.00, 'C'),
(1005, 101, 2, 2, 1, '2019-12-11', 1300.00, 'C'),
(1006, 102, 3, 3, 2, '2019-12-11', 1400.00, 'C'),
(1007, 103, 4, 1, 1, '2019-12-12', 1600.00, 'C'),
(1008, 104, 1, 2, 2, '2019-12-12', 1700.00, 'C');

-----------------------------------------------
-- Q1.1 TRAIN DETAILS
-----------------------------------------------

SELECT train_name, depart_time, arrival_time
FROM Train;

-----------------------------------------------
-- Q1.2 PASSENGERS AGE > 60
-----------------------------------------------

SELECT passenger_name, address
FROM Passenger
```

```
WHERE age > 60;

-----------------------------------------------
-- Q1.3 TRAINS WITH >10 BOGIES
-----------------------------------------------

SELECT source_stn, dest_stn
FROM Train
WHERE no_of_res_bogies > 10;

-----------------------------------------------
-- Q1.4 MALE PASSENGERS
-----------------------------------------------

SELECT passenger_name
FROM Passenger
WHERE gender = 'M';

-----------------------------------------------
-- Q2.1 TRIGGER: BOGIE CAPACITY <= 30
-----------------------------------------------

CREATE OR REPLACE FUNCTION check_bogie_capacity()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
```

```
    IF NEW.bogie_capacity > 30 THEN
        RAISE EXCEPTION 'Bogie capacity cannot exceed 30';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_bogie_capacity ON Train;

CREATE TRIGGER trg_bogie_capacity
BEFORE INSERT OR UPDATE ON Train
FOR EACH ROW
EXECUTE FUNCTION check_bogie_capacity();

-----------------------------------------------
-- Q2.2 FUNCTION: TOTAL TICKET AMOUNT (12-12-2019)
-----------------------------------------------

CREATE OR REPLACE FUNCTION total_ticket_amount()
RETURNS DECIMAL(10,2)
LANGUAGE plpgsql
AS $$
DECLARE
    total_amt DECIMAL(10,2);
BEGIN
    SELECT SUM(ticket_amt)
```

```
    INTO total_amt
    FROM Ticket
    WHERE tdate = '2019-12-12';

    RETURN COALESCE(total_amt, 0);
END;
$$;

-----------------------------------------------
-- FUNCTION CALL
-----------------------------------------------

SELECT total_ticket_amount();
```

Slip 27

Q.1) Consider the following database of Movie, Actor and Producers, [Total Marks: 10]

Movie (m_name varchar (25), release_year integer, budget money)

Actor (a_name char (30), city varchar(30))

Producer (producer_id integer, pname char (30), p_address varchar (30))

Relationship: Movie–Actor Relationship (Many-to-Many)

Movie–Producer Relationship (Many-to-One)

1. List all movies released in the year 2025.

2. Display the names of all actors who live in 'Delhi'.

3. List the name and budget of movies greater than 100 core.

4. List the names and addresses of all producers.

Q.2) Using above database solve following questions: [Total Marks:20]

1. Write a trigger before inserting record into movie table; check release_year should not be greater than

current year. Display appropriate message. [10]

2. Write a cursor using function to list movie-wise charges of ‘Madhuri Dixit’. [10]

Answer:

```
-- ============================

-- TABLE CREATION

-- ============================


CREATE TABLE Producer (

    producer_id INTEGER PRIMARY KEY,

    pname VARCHAR(30),
```

```
    p_address VARCHAR(30)
);

CREATE TABLE Movie (
    m_name VARCHAR(25) PRIMARY KEY,
    release_year INTEGER,
    budget NUMERIC,
    producer_id INTEGER,
    FOREIGN KEY (producer_id) REFERENCES Producer(producer_id)
);

CREATE TABLE Actor (
    a_name VARCHAR(30) PRIMARY KEY,
    city VARCHAR(30)
);

-- Many-to-Many relationship
CREATE TABLE Movie_Actor (
    m_name VARCHAR(25),
    a_name VARCHAR(30),
    charges NUMERIC,
    PRIMARY KEY (m_name, a_name),
    FOREIGN KEY (m_name) REFERENCES Movie(m_name),
    FOREIGN KEY (a_name) REFERENCES Actor(a_name)
);

-- ============================
```

```
-- INSERT DATA
-- ==========================

INSERT INTO Producer VALUES
(1, 'Karan Johar', 'Mumbai'),
(2, 'Rohit Shetty', 'Mumbai'),
(3, 'Aditya Chopra', 'Delhi');

INSERT INTO Movie VALUES
('MovieA', 2025, 150000000, 1),
('MovieB', 2024, 90000000, 2),
('MovieC', 2025, 200000000, 3),
('MovieD', 2023, 80000000, 1);

INSERT INTO Actor VALUES
('Madhuri Dixit', 'Mumbai'),
('Shah Rukh Khan', 'Delhi'),
('Alia Bhatt', 'Mumbai'),
('Ranveer Singh', 'Delhi');

INSERT INTO Movie_Actor VALUES
('MovieA', 'Madhuri Dixit', 5000000),
('MovieA', 'Shah Rukh Khan', 7000000),
('MovieB', 'Alia Bhatt', 4000000),
('MovieB', 'Ranveer Singh', 4500000),
('MovieC', 'Madhuri Dixit', 6000000),
('MovieC', 'Alia Bhatt', 5500000),
```

('MovieD', 'Ranveer Singh', 3000000),

('MovieD', 'Shah Rukh Khan', 6500000);

-- ============================

-- Q1 QUERIES

-- ============================

-- 1. Movies released in 2025

SELECT m_name

FROM Movie

WHERE release_year = 2025;

-- 2. Actors living in Delhi

SELECT a_name

FROM Actor

WHERE city = 'Delhi';

-- 3. Movies with budget > 100 crore (10 crore = 100000000 approx unit assumed)

SELECT m_name, budget

FROM Movie

WHERE budget > 100000000;

-- 4. Producer details

SELECT pname, p_address

FROM Producer;

-- ============================

```
-- Q2 (1) TRIGGER
-- ==========================

CREATE OR REPLACE FUNCTION check_release_year()
RETURNS TRIGGER AS $$
BEGIN
   IF NEW.release_year > EXTRACT(YEAR FROM CURRENT_DATE)
THEN
      RAISE EXCEPTION 'Release year cannot be greater than current year';
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_release_year
BEFORE INSERT ON Movie
FOR EACH ROW
EXECUTE FUNCTION check_release_year();

-- ==========================
-- Q2 (2) CURSOR FUNCTION
-- ==========================

CREATE OR REPLACE FUNCTION madhuri_movie_charges()
RETURNS VOID AS $$
DECLARE
   rec RECORD;
   cur CURSOR FOR
```

```
        SELECT m_name, charges
        FROM Movie_Actor
        WHERE a_name = 'Madhuri Dixit';
BEGIN
    OPEN cur;

    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Movie: %, Charges: %',
            rec.m_name, rec.charges;
    END LOOP;

    CLOSE cur;
END;
$$ LANGUAGE plpgsql;

-- CALL FUNCTION
SELECT madhuri_movie_charges();
```

Slip 28

Q.1) Practical Questions on PostgresSQL [Total Marks: 10]

Consider the following database of Student, Course and Enrollment,

Student(student_id, name ,age)

Course (course_id , course_name, credits)

Enrollment (student_id, course_id, enrolment_date)

Relationship: Student–Enrollment (One-to-Many)

Course–Enrollment (One-to-Many)

Many-to-Many via Enrollment table

1. Insert 2 records into student table.

2. Show all courses with more than 3 credits.

3. Display the name of student whose age >20.

4. Display student id of B.C.A course.

Q.2) Using above database solve following questions: [Total Marks :20]

1. Write a function to display detail of student whose name start with letter 'P'. [10]

2. Create a view that shows each student's name and the courses they are enrolled in. [10]

Answer:

```
-- ============================
-- TABLE CREATION
-- ============================

CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    name VARCHAR(30),
```

```
    age INT
);

CREATE TABLE Course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(30),
    credits INT
);

CREATE TABLE Enrollment (
    student_id INT,
    course_id INT,
    enrolment_date DATE,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES Student(student_id),
    FOREIGN KEY (course_id) REFERENCES Course(course_id)
);

-- ============================
-- INSERT DATA
-- ============================

INSERT INTO Student VALUES
(1, 'Priya', 21),
(2, 'Amit', 19),
(3, 'Pooja', 22),
(4, 'Rahul', 23);
```

```
INSERT INTO Course VALUES
(101, 'BCA', 4),
(102, 'BBA', 3),
(103, 'MCA', 5),
(104, 'BSc IT', 4);

INSERT INTO Enrollment VALUES
(1, 101, '2026-01-10'),
(2, 102, '2026-01-11'),
(3, 103, '2026-01-12'),
(4, 101, '2026-01-13'),
(1, 104, '2026-01-14'),
(2, 101, '2026-01-15'),
(3, 102, '2026-01-16'),
(4, 103, '2026-01-17');

-- Extra 2 Student records
INSERT INTO Student VALUES
(5, 'Pankaj', 24),
(6, 'Neha', 20);

-- ============================
-- Q1 QUERIES
-- ============================

-- 1. Courses with more than 3 credits
```

```
SELECT *
FROM Course
WHERE credits > 3;

-- 2. Students with age > 20
SELECT name
FROM Student
WHERE age > 20;

-- 3. Student IDs enrolled in BCA course
SELECT e.student_id
FROM Enrollment e
JOIN Course c ON e.course_id = c.course_id
WHERE c.course_name = 'BCA';

-- ============================
-- Q2 (1) FUNCTION
-- ============================

CREATE OR REPLACE FUNCTION students_starting_with_p()
RETURNS TABLE (
    student_id INT,
    name VARCHAR,
    age INT
)
LANGUAGE plpgsql
AS $$
```

```
BEGIN
    RETURN QUERY
    SELECT s.student_id, s.name, s.age
    FROM Student s
    WHERE s.name LIKE 'P%';
END;
$$;

-- Call function
SELECT * FROM students_starting_with_p();

-- ============================
-- Q2 (2) VIEW
-- ============================

CREATE VIEW student_course_view AS
SELECT s.name AS student_name,
       c.course_name
FROM Student s
JOIN Enrollment e ON s.student_id = e.student_id
JOIN Course c ON e.course_id = c.course_id;

-- View output
SELECT * FROM student_course_view;
```

Slip 29

Q.1) Practical Questions on PostgresSQL [Total Marks: 10]

Consider the following database of Bus, Route and Driver,

Bus (bus_no int ,b_capacity int , depot_name varchar(20))

Route (route_no int, source char (20), destination char (20), no_of_stations int)

Driver (driver_no int ,driver_name char(20), license_no int, address char(20), d_age int , salary float)

Relationship: Bus – Driver (One-to-One or One-to-Many)

Bus – Route (Many-to-One)

1. Display all buses with a capacity greater than 50

2. List all routes starting from 'Delhi'

3. Display names and license numbers of all drivers earning more than 25,000

4. List all the drivers of kothrud bus depot.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Write a stored function to accept route no and display bus information running on that route. [10]

2. Write a trigger which will prevent deleting drivers living in ‘Satara’. [10]

Answer:

```
-- ============================
-- TABLE CREATION
-- ============================

CREATE TABLE Route (
   route_no INT PRIMARY KEY,
   source VARCHAR(20),
```

```
    destination VARCHAR(20),
    no_of_stations INT
);

CREATE TABLE Bus (
    bus_no INT PRIMARY KEY,
    b_capacity INT,
    depot_name VARCHAR(20),
    route_no INT,
    FOREIGN KEY (route_no) REFERENCES Route(route_no)
);

CREATE TABLE Driver (
    driver_no INT PRIMARY KEY,
    driver_name VARCHAR(20),
    license_no INT,
    address VARCHAR(20),
    d_age INT,
    salary FLOAT,
    bus_no INT,
    FOREIGN KEY (bus_no) REFERENCES Bus(bus_no)
);

-- ============================
-- INSERT DATA
-- ============================
```

```
INSERT INTO Route VALUES
(1, 'Delhi', 'Agra', 10),
(2, 'Pune', 'Mumbai', 8),
(3, 'Delhi', 'Jaipur', 12),
(4, 'Nagpur', 'Pune', 15);

INSERT INTO Bus VALUES
(101, 60, 'Kothrud', 1),
(102, 45, 'ShivajiNagar', 2),
(103, 55, 'Kothrud', 3),
(104, 70, 'Swargate', 4);

INSERT INTO Driver VALUES
(1, 'Amit', 1111, 'Satara', 35, 30000, 101),
(2, 'Neha', 2222, 'Pune', 30, 26000, 102),
(3, 'Rahul', 3333, 'Mumbai', 40, 20000, 103),
(4, 'Sneha', 4444, 'Satara', 28, 27000, 104),
(5, 'Pooja', 5555, 'Pune', 32, 28000, 101),
(6, 'Rohan', 6666, 'Kothrud', 45, 35000, 102),
(7, 'Kiran', 7777, 'Satara', 38, 24000, 103),
(8, 'Anil', 8888, 'Kothrud', 50, 40000, 104);

-- ============================
-- Q1 QUERIES
-- ============================

-- 1. Buses with capacity > 50
```

```
SELECT *
FROM Bus
WHERE b_capacity > 50;

-- 2. Routes starting from Delhi
SELECT *
FROM Route
WHERE source = 'Delhi';

-- 3. Drivers earning more than 25000
SELECT driver_name, license_no
FROM Driver
WHERE salary > 25000;

-- 4. Drivers of Kothrud depot
SELECT d.driver_name
FROM Driver d
JOIN Bus b ON d.bus_no = b.bus_no
WHERE b.depot_name = 'Kothrud';

-- ============================
-- Q2 (1) FUNCTION
-- ============================

CREATE OR REPLACE FUNCTION get_bus_by_route(rno INT)
RETURNS TABLE (
    bus_no INT,
```

```
    b_capacity INT,
    depot_name VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT b.bus_no, b.b_capacity, b.depot_name
    FROM Bus b
    WHERE b.route_no = rno;
END;
$$;

-- Call function
SELECT * FROM get_bus_by_route(1);

-- ============================
-- Q2 (2) TRIGGER
-- ============================

CREATE OR REPLACE FUNCTION prevent_delete_satara()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.address = 'Satara' THEN
        RAISE EXCEPTION 'Cannot delete drivers from Satara';
    END IF;
    RETURN OLD;
```

```
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_delete_satara
BEFORE DELETE ON Driver
FOR EACH ROW
EXECUTE FUNCTION prevent_delete_satara();
```

Slip 30

Q.1) Practical Questions on PostgresSQL [Total Marks: 10]

Consider the following database of Branch, Customer and Loan_Application,

Branch (bid integer, brname char (30), brcity char (10))

Customer (cno integer, cname char (20), caddr char (35), city (20))

Loan_Application (lno integer, lamtrequired money, lamtapproved money, l_date date)

Relationship: Customer & Branch linked via Loan_Application (many-to-one each)

1. List the names of the customers who have received loan amount 1 lakh.

2. Find the maximum loan amount approved.

3. Find out the total loan amount sanctioned by "Deccan "branch.

4. Count the number of loan applications received by "J.M.Road" branch.

Q.2) Using above database solve following questions: [Total Marks: 20]

1. Create a view to display the details of all customers who have received loan amount less than

their requirement. [10]

2. Write a stored function to display customer details who have applied for loan more than one

branches. [10]

Answer:

```
-- ============================

-- TABLES

-- ============================


CREATE TABLE Branch (
```

```
    bid INTEGER PRIMARY KEY,
    brname VARCHAR(30),
    brcity VARCHAR(10)
);

CREATE TABLE Customer (
    cno INTEGER PRIMARY KEY,
    cname VARCHAR(20),
    caddr VARCHAR(35),
    city VARCHAR(20)
);

CREATE TABLE Loan_Application (
    lno INTEGER PRIMARY KEY,
    cno INTEGER REFERENCES Customer(cno),
    bid INTEGER REFERENCES Branch(bid),
    lamtrequired NUMERIC(12,2),
    lamtapproved NUMERIC(12,2),
    l_date DATE
);

-- ============================
-- DATA
-- ============================

INSERT INTO Branch VALUES
(1, 'Deccan', 'Pune'),
```

```
(2, 'J.M.Road', 'Pune'),
(3, 'ShivajiNagar', 'Pune');

INSERT INTO Customer VALUES
(101, 'Amit', 'Pune Camp', 'Pune'),
(102, 'Neha', 'Kothrud', 'Pune'),
(103, 'Rahul', 'Baner', 'Pune'),
(104, 'Sneha', 'Aundh', 'Pune');

INSERT INTO Loan_Application VALUES
(1, 101, 1, 150000, 100000, '2026-01-10'),
(2, 102, 2, 200000, 150000, '2026-02-12'),
(3, 103, 1, 120000, 100000, '2026-03-15'),
(4, 104, 3, 180000, 160000, '2026-03-20'),
(5, 101, 2, 130000, 100000, '2026-04-01'),
(6, 102, 1, 220000, 200000, '2026-04-05'),
(7, 103, 2, 140000, 120000, '2026-04-10'),
(8, 104, 1, 250000, 100000, '2026-04-15');

-- ============================
-- Q1 QUERIES
-- ============================

-- 1. Customers who got exactly 1 lakh
SELECT DISTINCT c.cname
FROM Customer c
JOIN Loan_Application l ON c.cno = l.cno
```

```
WHERE l.lamtapproved = 100000;

-- 2. Maximum loan approved
SELECT MAX(lamtapproved) AS max_loan
FROM Loan_Application;

-- 3. Total loan by Deccan branch
SELECT SUM(l.lamtapproved) AS total_loan
FROM Loan_Application l
JOIN Branch b ON l.bid = b.bid
WHERE b.brname = 'Deccan';

-- 4. Count applications for J.M.Road
SELECT COUNT(*) AS total_applications
FROM Loan_Application l
JOIN Branch b ON l.bid = b.bid
WHERE b.brname = 'J.M.Road';

-- ============================
-- VIEW
-- ============================

CREATE OR REPLACE VIEW vw_less_loan AS
SELECT
    c.cno,
    c.cname,
    l.lamtrequired,
```

```
    l.lamtapproved
FROM Customer c
JOIN Loan_Application l ON c.cno = l.cno
WHERE l.lamtapproved < l.lamtrequired;

SELECT * FROM vw_less_loan;

-- ============================
-- FUNCTION (FIXED SAFE VERSION)
-- ============================

CREATE OR REPLACE FUNCTION multi_branch_customers()
RETURNS TABLE (
    cno INTEGER,
    cname VARCHAR,
    branch_count INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        c.cno,
        c.cname,
        COUNT(DISTINCT l.bid)::INTEGER
    FROM Customer c
    JOIN Loan_Application l ON c.cno = l.cno
```

```
    GROUP BY c.cno, c.cname
    HAVING COUNT(DISTINCT l.bid) > 1;
END;
$$;

-- Call function
SELECT * FROM multi_branch_customers();
```

****

Note :-

We use these commands to completely reset PostgreSQL's working space so we can rebuild the database from scratch without errors.

```
DROP SCHEMA public CASCADE;
CREATE SCHEMA public;
```

Relationship Type	Number of Tables	Key Idea
One-to-One	2 (or 1)	Can merge or separate
One-to-Many	2	FK in many side
Many-to-One	2	Same as 1:M
Many-to-Many	3	Needs junction table